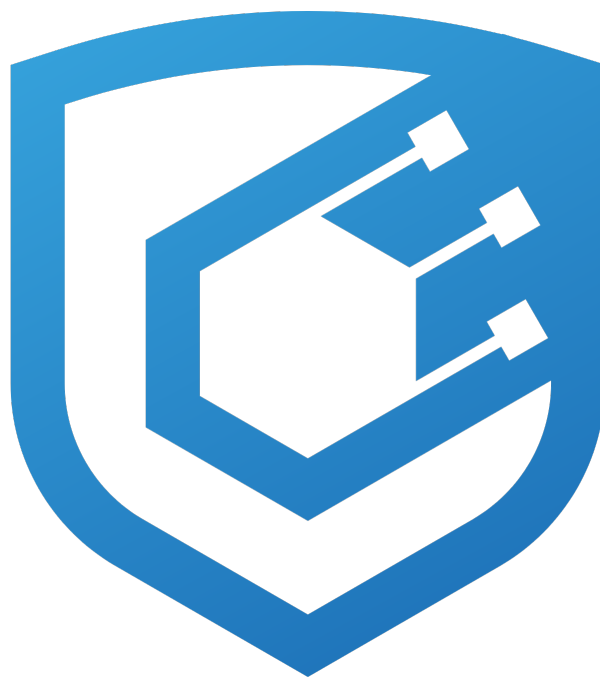




Highway EVM Audit Report

Prepared by [Cyfrin](#)

Version 2.1

Lead Auditors

[Farouk](#)

[qpzm](#)

September 29, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
4.1	Module / Component Overview	3
4.2	Message Lifecycle	4
4.3	Trust Model and Roles	4
4.4	Cross-Chain Consistency Model (the load-bearing invariant)	5
4.5	BLS12-381 Committee Attestation	5
4.6	Key Design Properties	5
5	Audit Scope	5
6	Executive Summary	6
7	Findings	11
7.1	High Risk	11
7.1.1	Non-injective MessageId SCALE encoding lets a payload-only source message execute as an unbacked token mint/release on the EVM	11
7.2	Medium Risk	19
7.2.1	ExecutorLogic::verifySignature resolves the inbound committee at claim time rather than the signing epoch	19
7.2.2	ActiveSetLogic::updateActiveSet does not validate that active-bitmap bits map to registered relayers	20
7.2.3	RelayerRegistryLogic::updateBlsKey rotates an active relayer's committee pubkey hash with no active-set guard	21
7.2.4	ExecutorLogic::executeMessage, executeMessageToken read corridor config live at claim time, not at signing time	22
7.2.5	executeMessage atomically couples token delivery to payload success with no failed-message recovery; a permanently-reverting payload strands the bridged tokens	23
7.2.6	BLS attestation has no per-network/genesis domain separation which allows cross-deployment replay	25
7.2.7	Source-chain sender is not bound into messageId, so a payload target cannot authenticate the source application	26
7.3	Low Risk	28
7.3.1	Whitelisting a protocol-internal contract lets MANAGE_WHITELIST_ROLE borrow the executor proxy's MANAGE_ESCROW_ROLE	28
7.3.2	ExecutorLogic::verifySignature active-set floor of 128 admits sizes where committee rejection-sampling deterministically exhausts	30
7.3.3	ExecutorLogic::executeMessage validates the converted amount before the amount>0 guard, making zero-token+payload messages permanently undeliverable	33
7.3.4	ActiveSetLogic::setEpochDuration, setRegistrationWindowBlocks retroactively recompute an effective set's in-flight epoch window	34
7.3.5	ExecutorLogic::verifyMessage BLS proof TTL has no upper bound relative to epoch duration	35
7.3.6	ExecutorLogic::_validateAndExecutePayload silently consumes a committed zero-target-address payload as a no-op	36
7.3.7	Storage, FeeCollector local pause does not halt bridge flow, leaving the emergency stop inconsistent across the storage contracts	36
7.3.8	Authorization rotation is impossible on the 4 logic proxies: the documented split-brain mitigation cannot be executed and any rotation permanently desyncs role and pause enforcement	37

7.3.9	ActiveSetLogic::updateActiveSet enforces only strict epoch monotonicity with no upper bound: a single over-large admin epoch permanently bricks the relay-driven deferred update path	38
7.3.10	RelayerRegistryStorage::updateMaxRelayerId is monotonic and never decreases on removal, permanently inflating the required active-set bitmap size after the highest-ID relay is removed	39
7.3.11	EntryLogic EIP-712 fee-quote domain separator hardcodes chainId=1 instead of block.chainid, permitting cross-EVM-chain replay if the proxy address and fee signer are reused	39
7.3.12	RelayerRegistryLogic::removeOperationalKey, addOperationalKey change submitter eligibility mid-flight without the active-or-pending guard their sibling carries	40
7.3.13	ExecutorLogic::_validateSourceChain does not reject originChainId == localChainId on a misconfigured self-origin corridor, permitting same-chain loopback messages	40
7.3.14	_countBits (SWAR popcount) returns 0 for type(uint256).max instead of 256	41
7.3.15	RelayerRegistryLogic::_verifyPop has no infinity guard	42
7.3.16	MessageId cannot encode Some(0) or Some(empty)	44
7.3.17	Outbound payload cap is not per-destination.	45
7.3.18	No rate limit, withdrawal delay, or circuit breaker bounds inbound damage	45
7.4	Informational	47
7.4.1	Storage::_validateBridgeConfiguration does not constrain per-corridor escrow to a single validated protocol escrow	47
7.4.2	ExecutorLogic::_handleMint does not reject a zero recipient for a MINT token that permits zero-address mints, burning bridged tokens to address(0) while the message is consumed	47
7.4.3	EntryLogic::emitMessage, emitMessageToken, emitMessagePayload do not validate destination-address byte-width against the target chain format, stranding burned or escrowed funds	48
7.4.4	ActiveSetLogic::updateActiveSet re-reads maxRelayerId at submission time: a concurrent registerRelayer forces a BitmapTooSmall revert on the deferred update path	49
7.4.5	5_SetupFeeCollection setup script is orphaned from every deployment path: the bridge goes live with fee validation disabled and zero fees collected	49
7.4.6	Deployment scripts silently fall back to the publicly-known Anvil account-0 private key when PRIVATE_KEY is unset, with no chain-id guard	50
7.4.7	Deployment scripts hardcode Highway LOCAL_CHAIN_ID=3 and Mosaic peer chainId=2 with no block.chainid assertion: a misdirected broadcast deploys a mis-keyed bridge	50
7.4.8	EntryLogic MessageEmitted omits sourceBlockHash, one of the committed inputs to the consensus-critical messageId	51
7.4.9	ActiveSetLogic ActiveSetUpdated emits only activeCount, not the full activeBitmap or maxRelayerId that defines committee membership	51
7.4.10	Unnamed non-aliased imports	52
7.4.11	Unused deprecated TokenConfig struct	52
7.4.12	aggregatePubkeys returns its single input unvalidated when n == 1, skipping the on-curve check it applies for n >= 2	53
7.5	Gas Optimization	55
7.5.1	Redundant hasTokenChainConfig external call before getTokenChainConfig	55
7.5.2	Cache the EntryLogic::_storage slot accessor instead of re-reading proxy slot on every call	55
7.5.3	Storage::getAndUpdateNonce increments storage then re-reads the same slot	56
7.5.4	Suboptimal storage packing on mapping-resident structs	56

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

Unless stated otherwise, runtime, deployment, and configuration values used in Cyfrin's analysis are those found in the reviewed code and materials at the audited commit. Later remediation values and Highway-provided operational values are identified as such. These values should not be read as those used by any production deployment, and Cyfrin did not independently verify any production deployment's configuration. This qualification does not affect conclusions based on a value identified in the report or a Highway response as fixed in the relevant code revision.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Highway (the Mosaic Bridge) is a cross-chain token-and-message bridge. This summary covers its EVM implementation (Ethereum), the audit scope; the bridge also operates Substrate/Polkadot and Solana legs that act as the EVM contracts' cross-chain counterparties. A message emitted on a source chain is attested by a BLS12-381 committee drawn from a shared relayer set, and executed on the destination chain, where it mints or releases tokens and/or dispatches a whitelisted payload call. A defining property of the system is that the message identity and the committee-attestation format must be reproduced byte-for-byte by each leg's independent implementation, so the EVM contracts must agree exactly with their counterparties on those encodings.

Production usage: every cross-chain transfer or message touches the full lifecycle. A user calls the source chain's emit entrypoint, which debits funds (burn or escrow), assigns a `messageId`, and emits the message. Off-chain relayers in the active committee observe it and produce a BLS aggregate attestation over a canonical payload. Any one committee relayer submits the attestation to the destination chain's execute entrypoint, which re-derives the `messageId`, verifies the aggregate against the committee it independently selects for that slot, marks the message processed (replay protection), and then mints/releases the tokens and dispatches the optional payload.

4.1 Module / Component Overview

Component	Role
EntryLogic	Outbound: emitMessage / emitMessageToken / emitMessagePayload; fee-quote (EIP-712) collection; burn/escrow
ExecutorLogic	Inbound: executeMessage*; BLS aggregate verification; deterministic committee selection; mint/release + payload dispatch
ActiveSetLogic / ActiveSetStorage	Active relay set, epoch ring buffer, registration window, epoch pinning
RelayerRegistryLogic / RelayerRegistryStorage	Relayer registration, BLS key + proof-of-possession, operational keys, registered-relayer bitmap
Storage / Escrow / FeeCollector / MessageStorage	Token & corridor config, escrowed funds, fee routing, processed-message replay set
PayloadWhitelist	(target, selector) allow-list for inbound payload dispatch
BLS12381 / BitmapUtils / MessageId	EIP-2537 BLS primitives + hash-to-curve, committee-bitmap helpers, canonical messageId encoder
Authorization / Authorized / proxies	Role-based access control, global pause, transparent proxies for the four logic contracts

4.2 Message Lifecycle

1. **Outbound (source).** The user calls an emit endpoint with a target chain, optional token leg (tokenId, target address, amount) and optional payload leg (target address, calldata), plus a signed fee quote. The source debits the funds (Burn or Escrow(account) mode), allocates a per-chain nonce, computes `messageId = H(domain ++ sourceChainId ++ targetChainId ++ sourceBlockHash ++ blockNumber ++ nonce ++ optional fields)`, and emits the message.
2. **Attestation (off-chain committee).** The relayers seated in the active committee for the message's slot co-sign a canonical BLS payload (`messageId ++ ttl ++ slotNumber ++ relayerId`) and produce a single aggregate signature plus a committee bitmap.
3. **Inbound (destination).** A committee relayer submits the attestation. The destination: checks replay (processed set), re-derives `messageId` from the supplied parameters and asserts equality, selects the committee deterministically from its own active set and epoch randomness, aggregates the bitmap-selected registered public keys on-chain, verifies the aggregate signature against a quorum threshold, marks the message processed, then performs the token leg (mint or release) and the payload leg (dispatch the whitelisted call).

4.3 Trust Model and Roles

- **Relayer committee (the security root).** Inbound execution trusts a threshold BLS aggregate from the active committee. The audited configuration used 128 seats with a threshold of 87, and Highway has since aligned the current threshold to 86 across the three implementations. Committee membership is selected deterministically per slot via keccak256 rejection sampling over the active set and epoch randomness, so the destination reconstructs exactly who must have signed.
- **Active set and epochs.** The active relay set is rotated by epoch; updates flow through an admin “immediate” path or a relayer/keeper “deferred” path constrained to a registration window. Each relayer registers a BLS key with a proof-of-possession before it can be seated.
- **Admin roles.** Authorization role-gating (DEFAULT_ADMIN_ROLE, ACTIVE_SET_ADMIN, MANAGE_STORAGE_ROLE, MANAGE_WHITELIST_ROLE, MANAGE_FEE_ROLE, MANAGE_ESCROW_ROLE, NONCE_MANAGER_ROLE) plus a

global pause.

- **Fee signer.** Outbound fees are authorized by an off-chain fee-signer over an EIP-712 fee quote; the on-chain path validates that signature before collecting the fee.

4.4 Cross-Chain Consistency Model (the load-bearing invariant)

Because a message is identified by a `messageId` that each chain recomputes independently, and because the committee attests a canonical payload that each chain independently reconstructs, the bridge is correct only if all three legs agree byte-for-byte on (a) the `messageId` encoding, (b) the BLS attestation payload, and (c) the committee-selection algorithm. The system is built to make (c) identical (a shared deterministic rejection sampling), and to share (a)/(b) across implementations. Most of the systemic risk in the protocol lives at this seam: any divergence in optional-field encoding, address width, domain separation, or signature ciphersuite causes the destination to compute a different `messageId`/aggregate and fail closed (a liveness/fund-stranding break), or, where it does not fail closed, opens a replay or substitution path. The encodings described here are the audited V1 (HWY_MSG_V1, no network domain, SCALE-option optional fields).

4.5 BLS12-381 Committee Attestation

Public keys are in G1 and signatures in G2 (minimal-pubkey-size); messages are hashed to G2 (`hashToG2`) via RFC 9380 SSWU_RO under the DST `BLS_SIG_BLS12381G2_XMD:SHA-256_SSWU_RO_NUL`; the aggregate is verified with a single pairing (FastAggregateVerify semantics) over committee keys aggregated on-chain. Rogue-key resistance is a proof-of-possession required at registration, though the audited DST uses the Basic `_NUL` tag while FastAggregateVerify is the proof-of-possession-scheme verification path. The EVM leg hand-implements the curve on the EIP-2537 precompiles.

4.6 Key Design Properties

- **Value conservation per corridor.** Each (token, chain) corridor is configured as `Burn/Mint` or `Escrow/Release`; the configuration enforces that lock and unlock (or destroy and create) stay paired, with the dangerous combinations (escrow-out + mint-in, burn-out + release-in) rejected.
- **Replay protection.** Each inbound `messageId` is recorded as processed before value moves, so a given message executes at most once per destination.
- **On-chain trustless aggregation.** The destination aggregates the committee public keys itself rather than accepting a submitted aggregate, and checks each supplied key against the stored registry.
- **Deterministic committee selection.** Committee seats are derived from epoch randomness, leader id, and slot via keccak256 rejection sampling, reproduced identically across legs so the attesting set and the verifying set coincide.
- **Fail-closed inbound.** Verification failures (wrong `messageId`, insufficient signers, unregistered key, paused) revert; a paused bridge or a divergent encoding stalls delivery rather than mis-delivering, with admin-gated recovery for stranded value.

5 Audit Scope

The audit scope was limited to:

```
src/access/Authorization.sol
src/access/Authorized.sol
src/libraries/BitmapUtils.sol
src/libraries/BLS12381.sol
src/libraries/BridgeConstants.sol
src/libraries/BridgeErrors.sol
src/libraries/BridgeEvents.sol
src/libraries/BridgeTypes.sol
src/libraries/MessageId.sol
```

```
src/logic/ActiveSetLogic.sol
src/logic/EntryLogic.sol
src/logic/ExecutorLogic.sol
src/logic/RelayerRegistryLogic.sol
src/proxy/ActiveSetProxy.sol
src/proxy/EntryLogicProxy.sol
src/proxy/ExecutorLogicProxy.sol
src/proxy/RelayerRegistryProxy.sol
src/storage/ActiveSetStorage.sol
src/storage/Escrow.sol
src/storage/FeeCollector.sol
src/storage/MessageStorage.sol
src/storage/PayloadWhitelist.sol
src/storage/RelayerRegistryStorage.sol
src/storage/Storage.sol
```

6 Executive Summary

Over the course of 15 days, the Cyfrin team conducted an audit on the [Highway EVM](#) code provided by [Highway](#). The findings consist of 1 High, 7 Medium & 18 Low severity issues with the remainder being informational and gas optimizations.

Cyfrin reviewed the EVM implementation of Highway (the Mosaic Bridge), a cross-chain token-and-message bridge secured by a BLS12-381 relayer committee. The review covered the EVM contracts: the outbound and inbound logic (EntryLogic, ExecutorLogic), the active-set and relayer registry (ActiveSetLogic, RelayerRegistryLogic), the Storage / Escrow / FeeCollector / MessageStorage / PayloadWhitelist stack, and the BLS12381 / BitmapUtils / MessageId libraries, behind their proxies. At risk are the funds escrowed or mint-controlled by these contracts and the integrity of the inbound messages that move them.

A message is identified by a `messageId` and authenticated by a committee attestation that each chain recomputes independently, so the EVM contracts are correct only when their encodings stay consistent with the rest of the system. We focused on where that agreement can break or where inbound execution can be steered away from what the committee attested: the `messageId` encoding (optional-field and address-width handling); the BLS attestation payload (domain separation, ciphersuite, proof-of-possession); the committee selection and its bitmap-to-key resolution; the inbound path that mints, releases, or dispatches on caller-supplied parameters; the corridor configuration that keeps lock and unlock paired; and the access-control and deployment surface that arms all of it.

The findings concentrate not in the signature construction, which is sound, but in inbound-delivery binding, encoding consistency, and the privilege surface of deployment and payload dispatch. The most prominent is a payload-dispatch confused-deputy: the inbound payload mechanism runs a whitelisted call with the executor proxy as `msg.sender`, and that proxy holds the bridge's own privileged roles, so a whitelisted protocol-internal target or bridge token could be driven to drain escrow or mint unbacked tokens. It is gated by the admin-controlled whitelist and a deploy-time role left on the deployer key, so it is a privilege-separation defect under a trusted-operator model, not a permissionless drain. Separately, the bridge does not authenticate the source-chain sender: the sender is not bound into the `messageId`, so any account can permissionlessly drive a whitelisted payload target that assumes a trusted source counterpart, with no committee collusion or misbehavior required. Highway acknowledged this as valid and is deferring it as a deliberate breaking change, since binding the sender alters every message hash and the target interface; until then the safety argument rests on the deny-by-default payload whitelist.

The largest cluster is encoding consistency. The audited V1 encoding had several issues: a High Risk non-injective `messageId` encoding that could let a payload-only source message collide with a token-bearing destination message; sentinel/Option mismatches that could strand otherwise valid messages; missing per-deployment domain separation in the BLS payload; an EIP-712 fee-quote domain hardcoded to `chainId = 1`; and a proof-of-possession DST/ciphersuite mismatch. The cross-deployment replay issue also required the destination deployment to resolve the same committee keys, active set, and randomness, but the signed bytes themselves were not deployment-bound. Highway addressed the message-ID, fee-quote, and network-domain items in EVM PR 126 while retaining the V1 identifiers: the current format binds an immutable per-deployment network ID into the relevant preimages, uses one canonical length-prefixed encoding, and binds the EIP-712 fee-quote domain to

block.chainid. EVM PR 112 added the attestation epoch. PR 97 closed without merge; PR 129 is its operative replacement, retaining the `_NUL_` ciphersuite for committee attestations and using the dedicated `_POP_` ciphersuite for proof of possession. Current Substrate and Solana production source reflects the same network-bound V1 message-ID and epoch-bound attestation application preimages, with Solana using curve-specific BN254 verification. End-to-end release parity is not established beyond the verified production code paths.

The remaining findings fall into four areas. **(1) Committee and threshold integrity and the relayer lifecycle:** the attestation pins the committee at claim time rather than the attested epoch, rejection sampling can deterministically exhaust near the 128 active-set floor, the active-set bitmap is not checked against registry membership, and relayer lifecycle changes can break committee determinism for in-flight proofs. **(2) BLS input hygiene:** the registration path validates only key length and accepts the point at infinity in proof-of-possession, with no on-curve, subgroup, or non-infinity check. **(3) Value-movement coupling and recovery:** the token and payload legs execute atomically with no recovery path when the payload leg reverts, and there is no per-destination payload-size limit, so the source accepts and charges for payloads a lower-capacity destination cannot deliver. **(4) Access control, bootstrap, and configuration:** deploy-time least privilege, epoch-zero and over-large-epoch bootstrap edge cases, pause coherence, and admin-key handling. Lower-severity findings cover defensive hardening, including the absence of any rate limit, withdrawal delay, or circuit breaker behind committee verification, so a single forged or colluded attestation has no aggregate damage bound beyond per-transaction caps and escrow balance.

Highway remediated actively during the engagement. The delivered changes cover the encoding and domain rework, proof-of-possession ciphersuite separation, fee-quote domain, payload-target validation, committee and epoch handling, the payload-whitelist protocol-internal guard, and several lower-severity items. Every EVM finding now carries a final report status: the remaining non-resolved findings are acknowledged, and no finding still points to an unmerged pull request. Cross-implementation format safety continues to depend on coordinated releases and compatibility testing.

The residual risk concentrates in two areas: binding inbound execution to the exact authenticated source context, and keeping the encodings consistent across the system. Recommendations: bind the source sender into the attested message and surface it to payload targets before integrations rely on source-application identity; maintain the adopted cross-implementation formats through coordinated releases and versioned compatibility tests; validate BLS keys at registration (on-curve, subgroup, non-infinity); add a recovery path for a failed payload leg; and consider a per-destination payload cap and a damage bound behind committee verification.

Summary

Project Name	Highway EVM
Repository	hway-ethereum
Commit	73def9a5bdc5...
Audit Timeline	June 8th - June 26th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	7	3	4
Low Risk	18	11	7
Informational	12	8	4
Gas Optimizations	4	4	0
Total	42	27	15

Summary of Findings

[H-1] Non-injective <code>MessageId</code> SCALE encoding lets a payload-only source message execute as an unbacked token mint/release on the EVM	Resolved
[M-1] <code>ExecutorLogic::verifySignature</code> resolves the inbound committee at claim time rather than the signing epoch	Resolved
[M-2] <code>ActiveSetLogic::updateActiveSet</code> does not validate that active-bitmap bits map to registered relayers	Resolved
[M-3] <code>RelayerRegistryLogic::updateBlsKey</code> rotates an active relayer's committee pubkey hash with no active-set guard	Acknowledged
[M-4] <code>ExecutorLogic::executeMessage</code> , <code>executeMessageToken</code> read corridor config live at claim time, not at signing time	Acknowledged
[M-5] <code>executeMessage</code> atomically couples token delivery to payload success with no failed-message recovery; a permanently-reverting payload strands the bridged tokens	Acknowledged
[M-6] BLS attestation has no per-network/genesis domain separation which allows cross-deployment replay	Resolved
[M-7] Source-chain sender is not bound into <code>messageId</code> , so a payload target cannot authenticate the source application	Acknowledged
[L-01] Whitelisting a protocol-internal contract lets <code>MANAGE_WHITELIST_ROLE</code> borrow the executor proxy's <code>MANAGE_ESCROW_ROLE</code>	Resolved
[L-02] <code>ExecutorLogic::verifySignature</code> active-set floor of 128 admits sizes where committee rejection-sampling deterministically exhausts	Acknowledged
[L-03] <code>ExecutorLogic::executeMessage</code> validates the converted amount before the <code>amount>0</code> guard, making zero-token+payload messages permanently undeliverable	Resolved
[L-04] <code>ActiveSetLogic::setEpochDuration</code> , <code>setRegistrationWindow</code> -Blocks retroactively recompute an effective set's in-flight epoch window	Acknowledged
[L-05] <code>ExecutorLogic::verifyMessage</code> BLS proof TTL has no upper bound relative to epoch duration	Resolved
[L-06] <code>ExecutorLogic::_validateAndExecutePayload</code> silently consumes a committed zero-target-address payload as a no-op	Resolved

[L-07] Storage, FeeCollector local pause does not halt bridge flow, leaving the emergency stop inconsistent across the storage contracts	Acknowledged
[L-08] Authorization rotation is impossible on the 4 logic proxies: the documented split-brain mitigation cannot be executed and any rotation permanently desyncs role and pause enforcement	Resolved
[L-09] ActiveSetLogic::updateActiveSet enforces only strict epoch monotonicity with no upper bound: a single over-large admin epoch permanently bricks the relayer-driven deferred update path	Resolved
[L-10] RelayerRegistryStorage::updateMaxRelayerId is monotonic and never decreases on removal, permanently inflating the required active-set bitmap size after the highest-ID relayer is removed	Acknowledged
[L-11] EntryLogic EIP-712 fee-quote domain separator hardcodes chainId=1 instead of block.chainid, permitting cross-EVM-chain replay if the proxy address and fee signer are reused	Resolved
[L-12] RelayerRegistryLogic::removeOperationalKey, addOperationalKey change submitter eligibility mid-flight without the active-or-pending guard their sibling carries	Acknowledged
[L-13] ExecutorLogic::_validateSourceChain does not reject originChainId == localChainId on a misconfigured self-origin corridor, permitting same-chain loopback messages	Resolved
[L-14] _countBits (SWAR popcount) returns 0 for type(uint256).max instead of 256	Resolved
[L-15] RelayerRegistryLogic::_verifyPop has no infinity guard	Resolved
[L-16] MessageId cannot encode Some(0) or Some(empty)	Resolved
[L-17] Outbound payload cap is not per-destination.	Acknowledged
[L-18] No rate limit, withdrawal delay, or circuit breaker bounds inbound damage	Acknowledged
[I-01] Storage::_validateBridgeConfiguration does not constrain per-corridor escrow to a single validated protocol escrow	Resolved
[I-02] ExecutorLogic::_handleMint does not reject a zero recipient for a MINT token that permits zero-address mints, burning bridged tokens to address(0) while the message is consumed	Resolved
[I-03] EntryLogic::emitMessage, emitMessageToken, emitMessagePayload do not validate destination-address byte-width against the target chain format, stranding burned or escrowed funds	Acknowledged
[I-04] ActiveSetLogic::updateActiveSet re-reads maxRelayerId at submission time: a concurrent registerRelayer forces a BitmapTooSmall revert on the deferred update path	Acknowledged
[I-05] 5_SetupFeeCollection setup script is orphaned from every deployment path: the bridge goes live with fee validation disabled and zero fees collected	Resolved
[I-06] Deployment scripts silently fall back to the publicly-known Anvil account-0 private key when PRIVATE_KEY is unset, with no chain-id guard	Resolved
[I-07] Deployment scripts hardcode Highway LOCAL_CHAIN_ID=3 and Mosaic peer chainId=2 with no block.chainid assertion: a misdirected broadcast deploys a mis-keyed bridge	Resolved

[I-08] EntryLogic MessageEmitted omits sourceBlockHash, one of the committed inputs to the consensus-critical messageId	Acknowledged
[I-09] ActiveSetLogic ActiveSetUpdated emits only activeCount, not the full activeBitmap or maxRelayerId that defines committee membership	Acknowledged
[I-10] Unnamed non-aliased imports	Resolved
[I-11] Unused deprecated TokenConfig struct	Resolved
[I-12] aggregatePubkeys returns its single input unvalidated when $n == 1$, skipping the on-curve check it applies for $n \geq 2$	Resolved
[G-1] Redundant hasTokenChainConfig external call before getTokenChainConfig	Resolved
[G-2] Cache the EntryLogic::_storage slot accessor instead of re-reading proxy slot on every call	Resolved
[G-3] Storage::getAndUpdateNonce increments storage then re-reads the same slot	Resolved
[G-4] Suboptimal storage packing on mapping-resident structs	Resolved

7 Findings

7.1 High Risk

7.1.1 Non-injective MessageId SCALE encoding lets a payload-only source message execute as an unbacked token mint/release on the EVM

Description: MessageId.generateMessageId (src/libraries/MessageId.sol:47-74) builds the consensus-critical message identifier by keccak256 over a fixed-width prefix followed by five **optional fields concatenated with no field tags and no separators**:

```
DOMAIN u32LE(sourceChainId) u32LE(targetChainId) sourceBlockHash(32) u64LE(blockNumber)
↳ u128LE(nonce)
|| _scaleOptionAddress(payloadTargetAddress)
|| _scaleOptionBytes(payload)
|| _scaleOptionU32(tokenId)
|| _scaleOptionAddress(tokenTargetAddress)
|| _scaleOptionU128(amount)
```

The only thing delimiting one optional field from the next is each encoder's own framing - and _scaleOptionAddress (src/libraries/MessageId.sol:95-103) is not self-delimiting, because it emits **two different forms under the same 0x01 tag**, with no type or length discriminant to tell them apart:

- a **32-byte** value → 0x01 || <32 raw bytes> (Substrate AccountId32 form, **no length prefix**);
- any other non-empty length → _scaleOptionBytes → 0x01 || compact(len) || bytes.

(_scaleOptionBytes itself, used for the payload field at src/libraries/MessageId.sol:80-89, is length-prefixed and therefore self-delimiting - the ambiguity is purely the address encoder's untyped 32-byte branch.)

Because the 32-byte address form emits **unframed** bytes and the five fields carry no tags, the field boundaries are not self-delimiting: two different field-tuples can serialize to the **same byte string**, so generateMessageId is **not injective** over the EVM message fields. This is distinct from the typed-Option parity class (which concerns source shapes the EVM cannot reproduce, causing rejection/stranding) and from the malformed-outbound self-stranding class - here the EVM reproduces one identical byte string for two *different* semantic messages, so verification passes and value is created.

A single preimage with two valid interpretations:

- **Source (payload-only)** - payloadTargetAddress = P (32 bytes), payload = S (opaque), tokenId = 0, tokenTargetAddress = empty set, amount = 0: 01 · P(32) · 01 compact(|S|) S · 00 · 00 · 00
- **Destination (token + payload)** - payloadTargetAddress = A (20-byte address), payload = D, tokenId = 12, tokenTargetAddress = R (20-byte recipient), amount = 1e18: 01 · 50 A(20) · 01 compact(|D|) D · 01 0c000000 · 01 50 R(20) · 01 amount(16) (compact(20) = 0x50)

The attacker chooses the source's 32-byte P to equal the destination's leading framing bytes 0x50 || A || 0x01 || compact(|D|) || D[0..], and packs the destination's tokenId / tokenTargetAddress / amount into the source's opaque length-prefixed payload S. Because P carries no length prefix, its 32 bytes silently absorb the destination's field framing, and the opaque payload swallows the destination's token tail. The two variable regions become byte-identical; the fixed prefix is identical because the destination submission reuses the same source-chain metadata (sourceBlockHash, blockNumber, nonce, chain ids) carried by the emitted source event - the attacker does not choose these values, only replays them into the destination call. Both interpretations therefore hash to the **same** messageId.

The destination accepts the collision because it **re-encodes** rather than decodes. executeMessage calls _verifyMessageId (src/logic/ExecutorLogic.sol), which recomputes MessageId.generateMessageId(originChainId, localChainId, sourceBlockHash, sourceBlockNumber, sourceNonce, payloadTargetAddress, payload, tokenId, tokenTargetAddress, amount) from the attacker-supplied typed fields and reverts only if messageId != expected. The reconstruction's chain order (originChainId, localChainId) mirrors the source's (sourceChainId = source, targetChainId = dest),

so the prefix matches, and the attacker-supplied token+payload fields hash to the attested payload-only messageId.

Every guard is upstream or downstream of the binding gap:

- `EntryLogic.emitMessagePayload` (`src/logic/EntryLogic.sol:457`) accepts bytes calldata payload-TargetAddress of arbitrary length and validates only active target chain, non-empty payload, payload max size, and fee quote - it never checks that the target width is canonical for the destination, so a 32-byte target is presentable on the source.
- The BLS attestation signs only `messageId || ttl || slotNumber || relayerId`, not the decoded token fields, so an **honest** committee attesting the honest payload-only source event produces an attestation that is equally valid for the token-bearing destination fields.
- The fee quote binds the source payload-only fields but is not part of the destination BLS message, so it cannot prevent a `messageId` collision once signed. A fee service *could* block this at emit by refusing a 32-byte EVM-bound payload target, but the on-chain protocol enforces no such canonical-width invariant - and when the fee signer is left unset, `_validateFeeQuote` early-returns (`src/logic/EntryLogic.sol:195-196`) so no off-chain gate runs at all.
- After `_verifyMessageId` passes, replay-marking, token-config lookup, amount-bounds, the mint/release helpers (`_handleMint` `src/logic/ExecutorLogic.sol:731`, `_handleRelease` :748), and the payload whitelist all run normally - the collision has already bypassed the only semantic binding between the attestation and the token fields.

Impact: A payload-only message is emitted on the source (permissionless apart from the normal fee) and burns or escrows no source token. The EVM destination then executes the same `messageId` as a token+payload message, minting a configured token (MINT corridor) or releasing escrowed liquidity (RELEASE corridor) to an attacker-controlled recipient, with the `tokenId`, recipient, and a **collision-compatible** amount chosen by the attacker - this layout admits any amount whose high bytes are zero (effectively unbounded for real token units), subject to the corridor's min/max bounds. This is mint/release with **no valid source message** for that token, recipient, and amount: the minted/released funds are real, the destination `messageId` is replay-marked after execution, and recovery requires governance/admin intervention outside the normal protocol path.

The attack does **not** require a dishonest threshold committee - the 87-of-128 signers honestly attest the real payload-only `messageId`. It does require a **single active relayer as the submitter** to present the colliding token+payload interpretation: `executeMessage` enforces `getRelayerIdByOperationalKey(msg.sender) == proof.relayerId` and that `proof.relayerId` is in the active set (`src/logic/ExecutorLogic.sol`), so the redeeming caller must be - or control - the active relayer named in the proof. One rogue or compromised relayer can therefore mint unbacked value, bypassing the committee's semantic intent; the source emit is permissionless, the destination redemption is not. The crafted token+payload message must also call a **whitelisted** (`target`, `selector`) **that succeeds**: `executeMessage` runs the mint/release (`src/logic/ExecutorLogic.sol:231-239`) before the payload call and is atomic, so a reverting payload (`PayloadCallFailed`) rolls back the mint. The attacker therefore needs a whitelisted function whose calldata tolerates the collision-crafted trailing bytes - the PoC uses a no-arg `ping()`, which Solidity lets ignore the extra bytes.

This finding, as written, is directly reachable only for an **EVM source** using this encoder - the PoC uses an `EntryLogicProxy` source, where `emitMessagePayload` accepts arbitrary-length target bytes, so a 32-byte `payloadTargetAddress` is presentable. If the only live source is Substrate/Mosaic, it is **latent unless the Substrate encoder can produce the same unframed 32-byte (AccountId32-native) address form** and its emit path admits an attacker-controlled target of that width.

Proof of Concept: The Foundry test below runs the exploit end-to-end through the bridge logic: it constructs the collision, emits the payload-only message on an independent source `EntryLogic` (which moves no tokens), then executes it on the destination as a token+payload message and asserts the recipient is **minted 1 ether despite the source event carrying no token fields**, and that the colliding id is replay-marked. It builds on the bridge's `G1BridgeFixture` harness (destination bridge core, active relayer set, the wired `MINT_TOKEN_ID` mint corridor, and `_buildProof()` for the committee attestation). The BLS path uses the standard local EIP-2537 precompile mock - this does not relax the exploit, because BLS signs only `messageId || ttl || slotNumber || relayerId` and both interpretations share the same `messageId`.

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.30;

import {EntryLogic} from "../../src/logic/EntryLogic.sol";
import {EntryLogicProxy} from "../../src/proxy/EntryLogicProxy.sol";
import {Storage} from "../../src/storage/Storage.sol";
import {Escrow} from "../../src/storage/Escrow.sol";
import {FeeCollector} from "../../src/storage/FeeCollector.sol";
import {Authorization} from "../../src/access/Authorization.sol";
import {BridgeConstants} from "../../src/libraries/BridgeConstants.sol";
import {BridgeEvents} from "../../src/libraries/BridgeEvents.sol";
import {MessageId} from "../../src/libraries/MessageId.sol";
import {G1BridgeFixture} from "../gl_token_conservation.t.sol";

contract CollisionPayloadTarget {
    bool public called;

    function ping() external {
        called = true;
    }
}

contract ExploreMessageIdPreimageCollisionTest is G1BridgeFixture {
    uint256 private constant FEE_SIGNER_PK = 0xA11CE;
    bytes32 private constant DOMAIN_TYPEHASH =
        keccak256("EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)");
    bytes32 private constant FEE_QUOTE_TYPEHASH = keccak256(
        "FeeQuote(address sender,uint32 targetChain,bytes32 targetAddressHash,bytes32
        ↪ targetPayloadAddressHash,uint32 tokenId,uint256 amount,bytes32 payloadHash,address
        ↪ feeToken,uint256 feeAmountExecution,uint256 feeAmountPlatform,uint256 ttl)"
    );
    bytes32 private constant EMPTY_ADDRESS_HASH =
        ↪ keccak256(bytes("0x0000000000000000000000000000000000000000000000000000000000000000"));

    Authorization private sourceAuthorization;
    Storage private sourceStorage;
    FeeCollector private sourceFeeCollector;
    EntryLogic private sourceEntry;

    function setUp() public {
        _deployBridgeCore();
        _setupRelayersAndActiveSet(address(this));
        _deploySourceEntry(CHAIN_B);

        CollisionPayloadTarget payloadTarget = new CollisionPayloadTarget();
        bytes4[] memory selectors = new bytes4[](1);
        selectors[0] = CollisionPayloadTarget.ping.selector;

        vm.prank(admin);
        authorization.grantRole(BridgeConstants.MANAGE_WHITELIST_ROLE, storageManager);
        vm.prank(storageManager);
        payloadWhitelist.registerContract(address(payloadTarget));
        vm.prank(storageManager);
        payloadWhitelist.addToWhitelist(address(payloadTarget), selectors);
    }

    function test_payloadOnlySourceMessageMintsAmountBearingDestinationMessage() public {
        CollisionPayloadTarget payloadTarget =
            ↪ CollisionPayloadTarget(payloadWhitelist.whitelistedContracts(0));

        uint128 nonce = 1;
        bytes memory destinationPayload =

```

```

        abi.encodePacked(CollisionPayloadTarget.ping.selector, bytes5(0xBEEFBEEFB1),
            ↪ bytes2(0x01A4));
    uint128 destinationAmount = uint128(1 ether);
    address destinationRecipient = recipient;

    bytes memory encodedDestinationOptions = _destinationOptions(
        address(payloadTarget), destinationPayload, MINT_TOKEN_ID, destinationRecipient,
        ↪ destinationAmount
    );
    assertEq(encodedDestinationOptions.length, 79, "expected compact collision layout");

    bytes memory sourcePayloadTarget = _slice(encodedDestinationOptions, 1, 32);
    bytes memory sourcePayload = _slice(encodedDestinationOptions, 35, 41);

    assertEq(encodedDestinationOptions[33], bytes1(0x01), "destination payload byte is source
    ↪ payload Some");
    assertEq(encodedDestinationOptions[34], bytes1(0xA4), "destination payload byte is compact len
    ↪ 41");
    assertEq(encodedDestinationOptions[76], bytes1(0x00), "source tokenId None");
    assertEq(encodedDestinationOptions[77], bytes1(0x00), "source token target None");
    assertEq(encodedDestinationOptions[78], bytes1(0x00), "source amount None");

    bytes32 sourceBlockHash = blockhash(block.number - 1);
    uint64 sourceBlockNumber = uint64(block.number);

    bytes32 sourceId = MessageId.generateMessageId(
        CHAIN_B,
        BridgeConstants.LOCAL_CHAIN_ID,
        sourceBlockHash,
        sourceBlockNumber,
        nonce,
        sourcePayloadTarget,
        sourcePayload,
        0,
        "",
        0
    );

    bytes32 destinationId = MessageId.generateMessageId(
        CHAIN_B,
        BridgeConstants.LOCAL_CHAIN_ID,
        sourceBlockHash,
        sourceBlockNumber,
        nonce,
        abi.encodePacked(address(payloadTarget)),
        destinationPayload,
        MINT_TOKEN_ID,
        abi.encodePacked(destinationRecipient),
        destinationAmount
    );

    assertEq(sourceId, destinationId, "same messageId covers two different semantic messages");
    assertEq(mintToken.balanceOf(destinationRecipient), 0, "recipient starts with no minted token");

    EntryLogic.FeeQuote memory sourceQuote =
        _signedSourceQuote(user, BridgeConstants.LOCAL_CHAIN_ID, "", sourcePayloadTarget, 0, 0,
            ↪ sourcePayload);

    vm.expectEmit(false, false, false, true, address(sourceEntry));
    emit BridgeEvents.MessageEmitted(
        sourceId,
        user,
        BridgeConstants.LOCAL_CHAIN_ID,

```

```

        sourcePayloadTarget,
        sourcePayload,
        "",
        0,
        0,
        block.number,
        nonce
    );

    vm.prank(user);
    sourceEntry.emitMessagePayload(BridgeConstants.LOCAL_CHAIN_ID, sourcePayloadTarget,
    ↪ sourcePayload, sourceQuote);

    assertEq(sourceStorage.getNonce(BridgeConstants.LOCAL_CHAIN_ID), nonce, "source emitted one
    ↪ message");

    // address(this) is registered as the active relayer's operational key in setUp
    // (via _setUpRelayersAndActiveSet), so this call passes the proof.relayerId identity gate.
    executor.executeMessage(
        destinationId,
        CHAIN_B,
        sourceBlockHash,
        sourceBlockNumber,
        nonce,
        address(payloadTarget),
        destinationPayload,
        destinationRecipient,
        _buildProof(),
        destinationAmount,
        MINT_TOKEN_ID
    );

    assertTrue(payloadTarget.called(), "normal whitelisted payload executed");
    assertEq(
        mintToken.balanceOf(destinationRecipient),
        destinationAmount,
        "destination minted even though source message had no token fields"
    );
    assertTrue(messageStorage.isMessageProcessed(destinationId), "colliding id was replay-marked");
}

function _deploySourceEntry(uint32 sourceLocalChainId) private {
    sourceAuthorization = new Authorization();
    sourceStorage = new Storage(address(sourceAuthorization));
    Escrow sourceEscrow = new Escrow(address(sourceAuthorization));
    sourceFeeCollector = new FeeCollector(address(sourceAuthorization));

    EntryLogicProxy sourceEntryProxy = new EntryLogicProxy(
        address(new EntryLogic(sourceLocalChainId)),
        admin,
        address(sourceAuthorization),
        address(sourceStorage),
        address(sourceEscrow),
        address(sourceFeeCollector),
        ""
    );
    sourceEntry = EntryLogic(address(sourceEntryProxy));

    sourceAuthorization.grantRole(BridgeConstants.MANAGE_STORAGE_ROLE, address(this));
    sourceAuthorization.grantRole(BridgeConstants.MANAGE_FEE_ROLE, address(this));
    sourceAuthorization.grantRole(BridgeConstants.NONCE_MANAGER_ROLE, address(sourceEntryProxy));

    sourceStorage.registerChain(BridgeConstants.LOCAL_CHAIN_ID, "Destination EVM", true);

```

```

        sourceFeeCollector.setExecutionRecipient(vm.addr(0xEEC));
        sourceFeeCollector.setPlatformRecipient(vm.addr(0xFEE));
        sourceFeeCollector.setFeeSigner(vm.addr(FEE_SIGNER_PK));
    }

    function _signedSourceQuote(
        address sender,
        uint32 targetChain,
        bytes memory targetAddress,
        bytes memory targetPayloadAddress,
        uint32 tokenId,
        uint256 amount,
        bytes memory payload
    ) private view returns (EntryLogic.FeeQuote memory) {
        uint256 ttl = block.number + 100;
        bytes32 targetAddressHash = targetAddress.length == 0 ? EMPTY_ADDRESS_HASH :
        ↪ keccak256(targetAddress);
        bytes32 targetPayloadAddressHash =
            targetPayloadAddress.length == 0 ? EMPTY_ADDRESS_HASH : keccak256(targetPayloadAddress);

        bytes32 domainSeparator = keccak256(
            abi.encode(
                DOMAIN_TYPEHASH,
                keccak256(bytes("Highway Bridge")),
                keccak256(bytes("1")),
                uint256(1),
                address(sourceEntry)
            )
        );
        bytes32 structHash = keccak256(
            abi.encode(
                FEE_QUOTE_TYPEHASH,
                sender,
                targetChain,
                targetAddressHash,
                targetPayloadAddressHash,
                tokenId,
                amount,
                keccak256(payload),
                address(0),
                uint256(0),
                uint256(0),
                ttl
            )
        );
        bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(FEE_SIGNER_PK, digest);

        return EntryLogic.FeeQuote({
            feeAmountExecution: 0,
            feeAmountPlatform: 0,
            feeToken: address(0),
            ttl: ttl,
            signature: abi.encodePacked(r, s, v)
        });
    }

    function _destinationOptions(
        address payloadTarget,
        bytes memory payload,
        uint32 tokenId,
        address tokenTarget,

```

```

        uint128 amount
    ) private pure returns (bytes memory) {
        return abi.encodePacked(
            bytes1(0x01),
            bytes1(0x50),
            abi.encodePacked(payloadTarget),
            bytes1(0x01),
            _scaleCompact(payload.length),
            payload,
            bytes1(0x01),
            _uint32LE(tokenId),
            bytes1(0x01),
            bytes1(0x50),
            abi.encodePacked(tokenTarget),
            bytes1(0x01),
            _uint128LE(amount)
        );
    }

function _slice(bytes memory data, uint256 start, uint256 len) private pure returns (bytes memory
    ↪ out) {
    out = new bytes(len);
    for (uint256 i = 0; i < len; i++) {
        out[i] = data[start + i];
    }
}

function _scaleCompact(uint256 value) private pure returns (bytes memory) {
    if (value < 64) return abi.encodePacked(uint8(value << 2));
    if (value < 16_384) {
        uint16 encoded = uint16((value << 2) | 0x01);
        return abi.encodePacked(uint8(encoded & 0xff), uint8(encoded >> 8));
    }
    if (value < 1_073_741_824) {
        uint32 encoded = uint32((value << 2) | 0x02);
        return abi.encodePacked(
            uint8(encoded & 0xff), uint8((encoded >> 8) & 0xff), uint8((encoded >> 16) & 0xff),
            ↪ uint8(encoded >> 24)
        );
    }
    revert("compact too large");
}

function _uint32LE(uint32 v) private pure returns (bytes4 result) {
    assembly {
        let r :=
            or(
                or(shl(24, and(v, 0xff)), shl(16, and(shr(8, v), 0xff))),
                or(shl(8, and(shr(16, v), 0xff)), and(shr(24, v), 0xff))
            )
        result := shl(224, r)
    }
}

function _uint128LE(uint128 v) private pure returns (bytes16 result) {
    assembly {
        let lo := and(v, 0xffffffffffffffff)
        let rlo :=
            or(
                or(
                    or(shl(56, and(lo, 0xff)), shl(48, and(shr(8, lo), 0xff))),
                    or(shl(40, and(shr(16, lo), 0xff)), shl(32, and(shr(24, lo), 0xff)))
                ),

```

```

        or(
            or(shl(24, and(shr(32, lo), 0xff)), shl(16, and(shr(40, lo), 0xff))),
            or(shl(8, and(shr(48, lo), 0xff)), and(shr(56, lo), 0xff))
        )
    )
    let hi := shr(64, v)
    let rhi :=
        or(
            or(
                or(shl(56, and(hi, 0xff)), shl(48, and(shr(8, hi), 0xff))),
                or(shl(40, and(shr(16, hi), 0xff)), shl(32, and(shr(24, hi), 0xff)))
            ),
            or(
                or(shl(24, and(shr(32, hi), 0xff)), shl(16, and(shr(40, hi), 0xff))),
                or(shl(8, and(shr(48, hi), 0xff)), and(shr(56, hi), 0xff))
            )
        )
    result := or(shl(192, rlo), shl(128, rhi))
}
}
}

```

```
forge test --match-test test_payloadOnlySourceMessageMintsAmountBearingDestinationMessage
```

Key assertions: `sourceId == destinationId` (the collision); the source emit moves no tokens (`emitMessagePayload`); and after `executeMessage`, `mintToken.balanceOf(destinationRecipient) == 1 ether` with the colliding id replay-marked - value minted with no token-bearing source message.

Recommended Mitigation: Make the `messageId` preimage prefix-free and typed so that distinct field-tuples can never serialize to the same bytes. Do not choose `AccountId32` vs `Vec<u8>` from raw byte length inside a generic encoder. Concretely:

- introduce a new message domain/version and encode every optional field with an explicit field tag, type tag, presence bit, and length (so each field is self-delimiting regardless of contents); or
- use chain-aware field encoders so EVM-bound address fields are always 20-byte length-prefixed values and Substrate account fields are typed `AccountId32` explicitly, never inferred from length; and
- reject unrepresentable or non-canonical outbound shapes before emitting - in particular a 32-byte EVM-bound `payloadTargetAddress`, and any target length other than the destination adapter's canonical width.

Because the preimage is recomputed identically on EVM and Substrate, any encoding change must land on both chains in lockstep. Add cross-field collision tests asserting that no payload-only message can share a `messageId` with any token-bearing message.

Highway: Fixed in [92cddcd](#) and [923bdeb](#).

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 `ExecutorLogic::verifySignature` resolves the inbound committee at claim time rather than the signing epoch

Description: Inbound execution is multi-step: a message is emitted on the source chain (debiting the user via burn or escrow), the active relayer committee aggregates a BLS signature off-chain, then a relayer submits `executeMessage`, `executeMessageToken`, or `executeMessagePayload` on Ethereum. The on-chain verifier `ExecutorLogic::verifySignature` (`src/logic/ExecutorLogic.sol:382-549`) reconstructs the committee that must have signed by reading `_activeSetProxy().getCurrentActiveSet()` (`src/logic/ExecutorLogic.sol:440`) - the active set in force at *claim time*, not the set the off-chain signers used. The `BlsProof` struct carries no epoch number (`src/libraries/BridgeTypes.sol:108-115`) and the BLS-signed payload commits no epoch (`messageId || ttl || slotNumber || relayerId`, `src/logic/ExecutorLogic.sol:424-428`), so verification cannot be pinned to the signing epoch. An in-code TODO at the seed step acknowledges this is deferred (`src/logic/ExecutorLogic.sol:460-461`), and `ActiveSetLogic::getActiveSetByEpoch` (`src/logic/ActiveSetLogic.sol:291-308`) - an epoch-indexed lookup over the depth-10 circular buffer - exists but is never consulted by the executor.

When a normal epoch rotation lands between signing and submission, three reads in `verifySignature` diverge from the signing context:

- `activeBitmap / nActive` flip to the new epoch's membership, so `BitmapUtils::buildPrefixSum`, `resolvePosition` map committee ordinals to different registry IDs;
- the submitting relayer's `relayerId` is checked against the new active bitmap (`src/logic/ExecutorLogic.sol:454-455`); a relayer active when signing but dropped from the new set reverts `RelayerNotInActiveSet`;
- `epochRandomness` flips to the new epoch's value, so `_computeCommitteeSeed` (`src/logic/ExecutorLogic.sol:462`) selects a different 128-seat committee.

The recomputed committee no longer matches the signers, so the call reverts `PubkeyHashMismatch / RelayerDoesNotExist` (per-seat hash check, `src/logic/ExecutorLogic.sol:527-529`) or `BLSVerificationFailed` (aggregate pairing, :547). The rotation is driven by `_resolveCurrentSet`, which flips the current set at the `block.number >= headSet.startBlock` pivot (`src/logic/ActiveSetLogic.sol:432`) once a pending set's future `startBlock = vEnd` is reached.

The failure is recoverable rather than terminal. All three entry points run `verifyMessage` (which calls `verifySignature`) before `markMessageAsProcessed` (`executeMessage :196→:213`, `executeMessageToken :266→:283`, `executeMessagePayload :329→:346`), so an epoch-mismatch revert leaves the `messageId` unprocessed and re-submittable. Because `messageId` is deterministic from source emission (reconstructed in `_verifyMessageId`), while `ttl`, `slotNumber`, and `relayerId` are proof-context fields, the current epoch-(N+1) committee can aggregate a fresh proof over the same `messageId` with refreshed proof parameters, and a relayer resubmits; verification then matches the current context and succeeds. Verification always requires a genuine `COMMITTEE_THRESHOLD-of-COMMITTEE_SIZE` aggregate from the current selected committee, so there is no forgery path under the honest-committee model.

The Substrate reference implements the identical claim-time resolution - `active_relayers_with_randomness()`, the same `is_active_relayer` membership check, and `select_committee(epoch_randomness, relayer_id, slot_number, ...)` - and carries a byte-identical TODO (`hwy-substrate/pallets/highway-entry/src/lib.rs:1135-1150`), so this is a cross-chain-consistent deferred design gap rather than an EVM-only divergence. Off-chain re-attestation is the practical recovery path in the current design.

Impact: Loss-of-availability for inbound messages whose off-chain signing-to-submission latency straddles an epoch boundary (epoch duration is approximately 1000 blocks / ~3.3h at 12s blocks, set in the out-of-scope deploy script, traced for context). Affected messages revert (`PubkeyHashMismatch / RelayerNotInActiveSet / BLSVerificationFailed`) and must be re-attested by the current committee and resubmitted, delaying settlement; the window recurs every epoch. All three inbound entry points are affected, since all route through `verifyMessage` → `verifySignature` and read the claim-time active set identically. For token transfers the source leg is already debited before destination execution, so a delayed message is a temporarily stuck, source-debited obligation; if it is not promptly re-attested, it may require out-of-band/source-side recovery (the README documents that recovery model for amount-derived inbound reverts, `README.md:222-225`). No funds are lost or forgeable, and permanent

stranding arises only if the off-chain network never re-attests, or the active set degrades below `COMMITTEE_SIZE` (`ActiveSetTooSmall` guard, `src/logic/ExecutorLogic.sol:443-445`).

Proof of Concept:

1. Epoch N is active. `getCurrentActiveSet` returns the epoch-N bitmap and randomness; the epoch-N committee is derived from (epoch-N randomness, `relayerId`, `slotNumber`).
2. A token-bridge message is emitted on the source chain; the source leg debits the user (burn or escrow). Off-chain, the epoch-N committee aggregates a BLS signature over `messageId || ttl || slotNumber || relayerId`; `proof.ttl` is comfortably in the future.
3. Before submission the epoch rotates: a pending epoch-(N+1) set with `startBlock = vEnd` reaches its `startBlock`, so `_resolveCurrentSet` flips at the `block.number >= headSet.startBlock` pivot (`src/logic/ActiveSetLogic.sol:432`) and `getCurrentActiveSet` now returns the epoch-(N+1) bitmap and randomness.
4. A relayer submits `executeMessageToken` with the epoch-N-signed proof. `verifyMessage` passes the replay check (`src/logic/ExecutorLogic.sol:363-365`) and TTL check (:368-370), then `verifySignature` reads the epoch-(N+1) values at :440: `_computeCommitteeSeed` over the new randomness (:462) yields a different committee, the resolved registry IDs differ, and the stored-pubkey-hash check (:527-529) reverts `Pubkey-HashMismatch` - or, if the submitter was dropped from the new bitmap, the membership check (:454-455) reverts `RelayerNotInActiveSet` first. The revert occurs before `markMessageAsProcessed` (:283), so `messageId` is not consumed.
5. The epoch-(N+1) committee re-aggregates a fresh proof over the same `messageId` (new `ttl / slotNumber / relayerId`, submitter a relayer active in epoch N+1) and resubmits; verification matches the epoch-(N+1) context and the message settles. Settlement is delayed by the rotation-window race. Permanent stranding arises only if the off-chain network never re-attests, or the active set degrades below `COMMITTEE_SIZE` (:443-445).

Recommended Mitigation: Add an epoch number (or active-set root commitment) to `BridgeTypes.BlsProof`, include it in the BLS-signed payload, and resolve the committee - bitmap, membership check, position resolution, and `epochRandomness` - from `getActiveSetByEpoch(proof.epoch)` instead of `getCurrentActiveSet()`. Bound acceptance to epochs still retained in the depth-10 circular buffer, reverting if the signing epoch has aged out. Keep the EVM and Substrate changes in lockstep to preserve the shared payload format. Note an adjacent interaction: pinning to historical sets means a relayer removed (and its pubkey hash deleted) after signing but before execution of a still-in-window message would strand it, so removal liveness should be modeled together with this fix. If the design is intentionally left as-is, document the operational expectation that relayers re-attest proofs stranded by a rotation and ensure the off-chain relayer client implements it.

Highway: Fixed in [40843cc](#).

Cyfrin: Verified.

7.2.2 `ActiveSetLogic::updateActiveSet` does not validate that active-bitmap bits map to registered relayers

Description: `ActiveSetLogic::updateActiveSet` validates a submitted active-set bitmap only for structural shape. `_validateBitmap` enforces the bitmap is exactly `ceil(maxRelayerId/8)` bytes (`src/logic/ActiveSetLogic.sol:549-552`) and that the trailing bits beyond `maxRelayerId` are zero (lines 556-561). It never verifies that each set bit corresponds to a relayer that is still registered in the registry.

Because relayer slots are deletable while `maxRelayerId` only ever increases, a bitmap can contain a phantom bit: a position for an ID that was removed or never registered. A phantom bit counts into `nActive` (the pop-count used as the committee-pool size). During inbound verification, the committee resolver maps each ordinal to a registry ID and reads its stored BLS key hash; if the resolved seat lands on a phantom position, `getRelayerPubkeyHash(rid)` returns `bytes32(0)` and `ExecutorLogic::verifySignature` reverts `RelayerDoesNotExist` (`src/logic/ExecutorLogic.sol:528`). Committee selection is deterministic in the epoch randomness, leader ID, and slot, so any message whose committee draws that phantom seat reverts identically on every retry for the life of the epoch.

This is the absence of a code-level membership check at bitmap-write time, not an admin-policy issue. The contract permits a structurally valid bitmap whose bits do not all map to live relayers; it should reject one.

Files:

- `ActiveSetLogic::updateActiveSet, _validateBitmap` - `src/logic/ActiveSetLogic.sol:543-562`

Impact: A phantom bit in an accepted active-set bitmap permanently blocks the class of inbound messages whose deterministic committee includes the phantom position, for the entire epoch. Affected messages cannot be delivered on the destination while their source-chain leg is already debited, leaving stranded value recoverable only via source-side admin recovery. Because relayer add/remove and active-set rotation are recurring operational events, the misconfiguration surface recurs each epoch. The inflated `nActive` also overstates the live committee pool relative to the BLS quorum threshold.

Recommended Mitigation: In `_validateBitmap` (or `updateActiveSet` before the set is written), iterate the set bits and verify each maps to a currently registered relayer via the registry's existence check, reverting on any phantom bit. This keeps `nActive` equal to the count of live committee-eligible relayers and prevents committee resolution from landing on a deleted slot.

Highway: Fixed in [d67077d](#).

Cyfrin: Verified.

7.2.3 `RelayerRegistryLogic::updateBlsKey` rotates an active relayer's committee pubkey hash with no active-set guard

Description: `RelayerRegistryLogic::removeRelayer` blocks removal while the relayer's bit is set in the current or pending active set (`isRelayerActiveOrPending`, reverting `RelayerStillActive` at `src/logic/RelayerRegistryLogic.sol:183-185`). Its sibling key-mutator `updateBlsKey` carries no such guard: it deletes the old BLS-key index and writes the new key hash (`src/logic/RelayerRegistryLogic.sol:220-222`) for any existing relayer, including one currently active in the committee.

Inbound verification reads the stored pubkey hash live. For each committee seat, `ExecutorLogic::verifySignature` resolves the seat to a registry ID and requires `keccak256(signerPubkey) == getRelayerPubkeyHash(rid)`, reverting `PubkeyHashMismatch` on a mismatch (`src/logic/ExecutorLogic.sol:526-530`). By design the committee's aggregate BLS signature is produced off-chain (outside this codebase) from the relayer keys current at signing time, while the on-chain check above reads `getRelayerPubkeyHash(rid)` live at claim time. Rotating a still-active relayer's BLS key changes that stored hash, so every in-flight message whose deterministic committee includes that relayer no longer verifies: the supplied signer pubkey no longer matches the rotated hash and the call reverts.

This is a code-level absence of enforcement, not an operator-discretion issue: the contract permits a BLS-key rotation on an active relayer that the sibling removal path explicitly forbids in the same active-set state. Admin sequencing is a mitigation, not a fix.

Files:

- `RelayerRegistryLogic::updateBlsKey, removeRelayer` - `src/logic/RelayerRegistryLogic.sol:177-225`
- `ExecutorLogic::verifySignature` - `src/logic/ExecutorLogic.sol:526-530`

Impact: A BLS-key rotation on an active relayer permanently strands every in-flight signed message whose committee includes that relayer until the new committee re-attests. For token transfers the source-chain leg is already debited (burned or escrowed), so value is stranded with no inbound settlement path, recoverable only via source-side admin recovery. Key rotation and message flow are recurring operational events, so the surface recurs.

Proof of Concept: Runnable Foundry PoC at [test/poc/M11_UpdateBlsKeyMidActive.t.sol](#). It registers relayer 1 with key `kOld`, seats it in the active set, then shows the asymmetry at the registry layer, `removeRelayer` is guarded against active relayers, `updateBlsKey` is not:

```
function test_updateBlsKey_onActiveRelayer_changesCommitteeKeyMidEpoch_noGuard() public {
    // Seat relayer 1 in the active set (admin path = immediate). Bitmap bit 0 = relayer 1.
    vm.prank(admin);
```

```

activeSet.updateActiveSet(1, hex"01", keccak256("epoch-1-randomness"));

// Relayer 1 is now active in the committee set.
assertTrue(activeSet.isRelayerActiveOrPending(1), "relayer 1 is active");

// (1) removeRelayer IS guarded: it refuses to mutate an active relayer.
vm.prank(relayerAdmin);
vm.expectRevert(abi.encodeWithSelector(BridgeErrors.RelayStillActive.selector, uint32(1)));
registry.removeRelayer(1);

// (2) updateBlsKey is NOT guarded: it succeeds while relayer 1 is active,
//      changing the committee key the contract verifies against.
bytes32 oldHash = registry.getRelayerPubkeyHash(1);
assertEq(oldHash, keccak256(kOld), "stored hash == old key");

vm.prank(relayerAdmin);
registry.updateBlsKey(1, kNew, pop); // no RelayStillActive guard -> succeeds mid-epoch

bytes32 newHash = registry.getRelayerPubkeyHash(1);
assertEq(newHash, keccak256(kNew), "stored hash now == new key");
assertTrue(oldHash != newHash, "committee key changed WHILE the relayer is active");

// Consequence (by inspection of ExecutorLogic.sol:526-529): verifyInboundProof reads
// getRelayerPubkeyHash(1) == newHash. An in-flight proof whose committee includes
// relayer 1, signed with kOld, hashes to oldHash != newHash -> PubkeyHashMismatch(1).
// The honest, already-signed attestation is invalidated mid-epoch - exactly the
// determinism break removeRelayer's guard prevents, but updateBlsKey does not.
}

```

[PASS] test_updateBlsKey_onActiveRelayer_changesCommitteeKeyMidEpoch_noGuard()

Recommended Mitigation: Guard `updateBlsKey` (and the operational-key mutators) with the same active-or-pending check `removeRelayer` uses, so a relayer's committee-verification material can only change while the relayer is inactive. Alternatively, bind the rotation to an explicit epoch transition so in-flight proofs continue to verify against the key that signed them.

Highway: Acknowledged; Assumption for this is that BLS key is changed because it was leaked, it is changed because it is a threat. Preventing BLS update would expose us to this threat where compromised keys can't be updated until new active set without the relayer needing update is out of the set.

Cyfrin: Rationale accepted with a condition. Immediate rotation of a compromised BLS key is a reasonable fail-closed security tradeoff, but the “no funds lost” conclusion depends on Highway reliably identifying and re-attesting every affected in-flight message under an accepted epoch; the EVM contracts preserve retryability but do not perform that resubmission.

7.2.4 `ExecutorLogic::executeMessage, executeMessageToken` read corridor config live at claim time, not at signing time

Description: Inbound execution reads the token and corridor config live at claim time. `ExecutorLogic::executeMessage, executeMessageToken` call `_storage().getTokenWithChainConfig(tokenId, originChainId)` after BLS verification (`src/logic/ExecutorLogic.sol:215-217`) and then use the returned values to gate the execution: `tokenInfo.enabled` (line 218), `chainConfig.sourceDecimals` and the resulting decimal conversion (line 221), `chainConfig.minAmount/maxAmount` (line 224), and the inbound bridge type and escrow.

Every one of these fields is mutable by `MANAGE_STORAGE_ROLE` between the moment the source chain signs the message and the moment a relayer submits it on Ethereum (`Storage::setTokenEnabled, updateChainStatus, removeTokenChainConfig, updateTokenBridgeLimits, configureBridge`). A mid-flight config change strands the debited source funds: disabling the token (`TokenNotEnabled`), removing the corridor, or narrowing the min/max bound makes the otherwise-valid inbound execution revert. Raising `sourceDecimals` makes the decimal conversion floor more aggressively, so the same attested source amount silently pays out less local value than intended.

The message was validly attested for the corridor config that existed at signing time. The contract binds settlement to the claim-time config with no snapshot of the signing-time config, so a routine config update reverts or short-pays an in-flight message.

Files:

- `ExecutorLogic::executeMessage, executeMessageToken` - `src/logic/ExecutorLogic.sol:215-224`

Impact: A corridor-config change while a message is in flight strands the message: its source-chain leg is already debited (burned or escrowed), the inbound execution reverts, and the value is recoverable only via source-side admin recovery. A `sourceDecimals` increase instead silently short-pays the recipient. Config changes and message flow are both recurring operational events, so the surface recurs whenever a corridor is reconfigured while transfers are in flight.

Recommended Mitigation: Bind inbound settlement to the corridor config that was in force when the message was attested rather than the live config. For example, include the relevant config parameters (decimals, bounds, bridge type) in the signed message and validate the on-chain config matches, or version corridor config and carry the version in the proof so a stale-but-valid in-flight message still settles. At minimum, document and constrain config mutation while transfers are in flight so disable/remove/narrow operations cannot strand attested messages.

Highway: Acknowledged; a live `sourceDecimals` change can allow an already-attested message to execute with a different payout and consume its replay ID. Before changing `sourceDecimals`, Highway will stop new source-side messages for the corridor, establish a cutoff, and reconcile every pre-cutoff message. We accept the Medium finding subject to this operational condition.

Cyfrin: Rationale accepted with a condition: before changing `sourceDecimals`, Highway must stop new source-side messages for the corridor, establish a cutoff, and reconcile every pre-cutoff message. This procedure is the compensating operational control; the code-level issue is not marked resolved.

7.2.5 `executeMessage` atomically couples token delivery to payload success with no failed-message recovery; a permanently-reverting payload strands the bridged tokens

Description: Highway's inbound executor is atomic, all-or-nothing. `ExecutorLogic.executeMessage` marks the message processed *before* effects, then transfers tokens, then runs the payload via `_validateAndExecutePayload`, which reverts on a failed inner call (the token handlers revert too). So any failure rolls back the whole tx including the processed-mark (`MessageStorage.sol:43`; the comment at ':227-230' says this is intentional): the message is not consumed and can be retried, but the tokens aren't delivered and there is no recovery primitive, no `failedMessages` queue, and no token-only fallback, since the `messageId` binds the `payloadHash`, so `executeMessageToken` reconstructs a different id and reverts `InvalidMessageId`.

This is the deliberate inverse of the OP Stack `L2CrossDomainMessenger` design: there, a failing inner call is *caught* and recorded in `failedMessages[hash]` for permissionless, never-expiring retry, and token bridging is a *separate* message from arbitrary calls, so a bad payload never blocks funds.

Impact: Severity is **Medium**. A single message that bridges tokens and carries a payload to a target that permanently reverts can never execute, and the bridged tokens are frozen with no permissionless / protocol-level recovery path (only governance-paced admin recovery, see below). Reaching the permanent-revert condition:

- The payload target is paused/self-destructed, or its logic reverts for the supplied args.
- The whitelisted (`target, selector`) is removed from the whitelist after the message was emitted, `validatePayload` then reverts, turning a third-party governance action into a stranding of a user's in-flight tokens.

In every such case the tokens sit in the source escrow (RELEASE) or remain unminted (MINT) with no way to claim them on the destination, `executeMessage` always reverts and `executeMessageToken` can't match the payload-bound `messageId`. There is no `failedMessages` queue and no token-only fallback to recover *on the protocol path*. The only recourse is governance-paced admin action: source-side `Escrow.recoverNative/recoverToken` (onlyAdmin; the code comments them as "recover funds from failed bridges") for RELEASE corridors, or an admin re-mint for MINT corridors (the source token was already burned). This is the same freeze-and-admin-recover posture in contrast to OP Stack's `failedMessages` which any party can re-relay permissionlessly and forever.

Proof of Concept: Runnable Foundry PoC at [test/poc/MsgRecovery_StuckTokensOnRevertingPayload.t.sol](#) (inherits the ExecutorLogic test harness; PASS). For a RELEASE+payload native-token message with a reverting target:

```
function test_PoC_revertingPayload_strandsBridgedTokens_noRecovery() public {
    uint256 amount = 1 ether;
    uint256 escrowBefore = address(escrow).balance;
    uint256 recipientBefore = _tokenRecipient().balance;

    // The payload target permanently reverts. (paused / bad-args / removed-whitelist all reduce to
    ↪ this.)
    payloadReceiver.setShouldRevert(true);

    // ONE inbound message bridges native tokens (RELEASE from escrow) AND carries a payload.
    bytes32 msgId = _msgIdMessage(42, address(payloadReceiver), payload, nativeTokenId, amount);
    BridgeTypes.BlsProof memory proof = _buildProof(msgId);

    // (1) Atomic revert: the failing payload reverts the entire execution.
    vm.expectRevert(BridgeErrors.PayloadCallFailed.selector);
    executorLogic.executeMessage(
        msgId, chainId2, TEST_BLOCK_HASH, TEST_BLOCK_NUMBER, 42,
        address(payloadReceiver), payload, testTokenTargetAddress, proof, amount, nativeTokenId
    );

    // (2) Nothing was delivered, and the message is NOT consumed (rollback).
    //     => no double-spend risk, but also nothing happened: the tokens stay in escrow.
    assertEq(address(escrow).balance, escrowBefore, "escrow untouched (tokens NOT released)");
    assertEq(_tokenRecipient().balance, recipientBefore, "recipient received nothing");
    assertFalse(messageStorage.isMessageProcessed(msgId), "message NOT consumed -> retryable, not
    ↪ marked");

    // (3) No token-only recovery: the bridged tokens cannot be claimed without the payload.
    //     The token-only entry reconstructs the messageId WITHOUT the payload, so it can never
    //     match the committee-attested tokens+payload messageId.
    bytes32 tokenOnlyId = _msgIdToken(42, nativeTokenId, amount);
    assertTrue(tokenOnlyId != msgId, "token-only id differs from tokens+payload id");
    vm.expectRevert(abi.encodeWithSelector(BridgeErrors.InvalidMessageId.selector, msgId, tokenOnlyId));
    executorLogic.executeMessageToken(
        msgId, chainId2, TEST_BLOCK_HASH, TEST_BLOCK_NUMBER, 42,
        testTokenTargetAddress, proof, amount, nativeTokenId
    );
    // While the payload reverts there is NO path to the tokens: no fallback, no failedMessages queue.

    // (4) Contrast: the stuck-ness is purely the payload. Fix the target and the SAME proof delivers,
    //     confirming the message was retryable (atomic) and only a *permanently*-reverting payload
    ↪ strands it.
    payloadReceiver.setShouldRevert(false);
    executorLogic.executeMessage(
        msgId, chainId2, TEST_BLOCK_HASH, TEST_BLOCK_NUMBER, 42,
        address(payloadReceiver), payload, testTokenTargetAddress, proof, amount, nativeTokenId
    );
    assertEq(_tokenRecipient().balance, recipientBefore + amount, "delivered once the target stops
    ↪ reverting");
    assertTrue(messageStorage.isMessageProcessed(msgId), "now consumed");
}
```

```
$ forge test --match-test test_PoC_revertingPayload_strandsBridgedTokens_noRecovery
[PASS] test_PoC_revertingPayload_strandsBridgedTokens_noRecovery()
```

Recommended Mitigation: Decouple token delivery from payload execution (the OP Stack shape). Deliver the tokens unconditionally, and execute the payload in a separate, independently-retryable step (catch the payload failure rather than reverting the token transfer). This removes the coupling entirely. If the all-or-nothing atomicity

is intended (so payloads can rely on the just-received tokens), document the invariant that a tokens+payload message to a target that may revert risks stranding funds, and ensure whitelist entries are never removed while messages referencing them may be in flight.

Highway: Acknowledged; It is intended behavior as sending tokens with payload should be trigger some coupled behavior with tokens and execution. Will add documentation comments.

Cyfrin: Rationale accepted with a condition: atomic token-and-payload execution is a reasonable design for payloads that depend on the delivered tokens, provided the published documentation clearly warns that a permanently reverting or de-whitelisted payload has no permissionless token-only fallback and requires governance recovery for both escrow/release and burn/mint corridors.

7.2.6 BLS attestation has no per-network/genesis domain separation which allows cross-deployment replay

Description: The BLS-signed message binds only `messageId ++ ttl_LE ++ slotNumber_LE ++ relayerId_LE` ([ExecutorLogic.sol:426-428](#)). The only chain identifier anywhere in those bytes is the logical Highway chain id carried inside `messageId`: `MessageId.generateMessageId` mixes `sourceChainId / targetChainId`, which on the EVM side is the constructor argument `localChainId`. That id is not the EVM chain id and is not per-deployment unique: [script/1_Deploy.s.sol:78-79](#) wires `BridgeConstants.LOCAL_CHAIN_ID = 3` ([BridgeConstants.sol:48](#)) into the `EntryLogic / ExecutorLogic` constructors for every deployment, and [script/README.md:182](#) confirms 3 is "the Highway-internal chain id ... not Sepolia's EVM chainId," shared across deployments of the same code (testnet, and mainnet).

Sharing the id across networks is not a documented design choice; it is the default behaviour of a hardcoded constant. `localChainId` is fixed at compile time, the deploy script never parameterises it per network (the only per-network ids `2_ConfigureBridge.s.sol` reads are the remote/target chains being registered, not this chain's own id), and the source comment treats 3 as the single canonical id for "this Ethereum deployment." So building and deploying the repository as-is to a testnet and then a mainnet yields `localChainId = 3` on both, unless an operator remembers to edit the constant between deployments, which nothing in the code or scripts prompts them to do.

There is no genesis hash, no EIP-155 `block.chainid` mixing, and no DST/domain suffix on the BLS hash-to-curve (the DST is the standard `BLS_SIG_BLS12381G2_XMD:SHA-256_SSWU_RO_NUL_`, [BLS12381.sol:54](#)) that distinguishes one Highway deployment from another.

Impact: Because the signed bytes carry no deployment-unique value, a committee attestation produced for one deployment is byte-identical to a valid attestation on another that shares `LocalChainId`. That byte-identity is the on-chain defect; whether a replayed attestation actually verifies on the second deployment additionally requires that the second deployment reproduces the **same committee** for the message. `verifySignature` aggregates the destination's *own* registered committee keys (selected from its active set via its own `epochRandomness`, [ExecutorLogic.sol:462 / :520-532](#)) and checks the signature against that aggregate ([:541-544](#)), so the replay succeeds only when the second deployment has (a) the same registered relayer keys, (b) the same active-set composition, and (c) the same `epochRandomness`, so the submitted bitmap resolves to the same keys. Bare key reuse alone is not sufficient; the whole committee selection must coincide.

The realistic scenario where all three hold is a clone or mirror: a mainnet bootstrapped from the testnet relayer set and state without rotating keys, or a staging environment seeded from production. In that case an attacker observing a testnet attestation can replay it on mainnet for the same logical message, getting tokens minted/released on the target side.

The risk depends on operational discipline (a freshly-randomized committee per deployment, ideally with distinct keys) which is not enforced on-chain.

Proof of Concept: Runnable Foundry PoC at [test/poc/M04_BlsNoNetworkDomain.t.sol](#) (passes, no precompiles / committee / key material needed). The test rebuilds the canonical BLS message exactly as `ExecutorLogic.verifySignature` does (`message = messageId ++ ttl_LE ++ slotNumber_LE ++ relayerId_LE`) over the real `MessageId.generateMessageId`, then shows that two EVM deployments which both use `LOCAL_CHAIN_ID = 3` produce byte-identical `messageId` and byte-identical signing bytes for the same logical inbound message under different `block.chainid`, so one committee signature is a valid signature on both

(`_blsMessage` mirrors `ExecutorLogic.sol:426-428`; `_uint32LE` mirrors the private `_uint32ToLE`; `_messageId` wraps the real `MessageId.generateMessageId`; constants are defined in the file):

```
// @dev Mirrors the canonical BLS message at ExecutorLogic.sol:426-428.
function _blsMessage(bytes32 messageId) internal pure returns (bytes memory) {
    return abi.encodePacked(messageId, _uint32LE(TTL), _uint32LE(SLOT), _uint32LE(RELAYER_ID));
}

function test_signingBytesIdentical_whenLocalChainIdShared() public {
    // Both EVM deployments run the standard deploy: localChainId = LOCAL_CHAIN_ID = 3.
    vm.chainId(SEPOLIA);
    bytes32 idTestnet = _messageId(BridgeConstants.LOCAL_CHAIN_ID);
    bytes memory msgTestnet = _blsMessage(idTestnet);

    vm.chainId(MAINNET);
    bytes32 idMainnet = _messageId(BridgeConstants.LOCAL_CHAIN_ID);
    bytes memory msgMainnet = _blsMessage(idMainnet);

    // messageId and the full BLS message are byte-identical across the two networks,
    // so one committee signature over msgTestnet is a valid signature over msgMainnet.
    assertEq(idTestnet, idMainnet, "messageId differs across deployments");
    assertEq(keccak256(msgTestnet), keccak256(msgMainnet), "BLS signing bytes differ across
    ↳ deployments");
}
```

[PASS] test_signingBytesIdentical_whenLocalChainIdShared()

A full on-chain replay (registering the same committee keys on two deployments and re-submitting one network's proof to the other's `executeMessage`) is the real-world manifestation; the PoC proves the load-bearing fact (the signed bytes carry no deployment-unique value) deterministically, which is what makes that replay succeed.

Recommended Mitigation: Bind a genuinely per-deployment value into the signed BLS message:

```
bytes memory message = abi.encodePacked(
    messageId, _uint32ToLE(proof.ttl), _uint32ToLE(proof.slotNumber), _uint32ToLE(proof.relayerId),
    DEPLOYMENT_DOMAIN // immutable, set at construction: e.g. keccak256("highway-v1") ++
    ↳ block.chainid
);
```

Applied identically on all chains, this prevents cross-deployment replay. The value must be per-deployment unique (genesis hash, deploy block hash, or `block.chainid`), not `localChainId`: that is the shared constant 3, so binding to it (as <https://github.com/Project-Highway/hway-ethereum/pull/104> does for the EIP-712 fee quote) does not separate two deployments of the same code. Highway already asserts the target id inside `messageId` via `_verifyMessageId`, so feeding that a per-deployment-unique id turns the existing check into a real separator with no new machinery.

Highway: Fixed in [92cddcd](#), [78ac6a6](#), [467221d](#) and [58aeb46](#).

Cyfrin: Verified.

7.2.7 Source-chain sender is not bound into `messageId`, so a payload target cannot authenticate the source application

Description: `MessageId.generateMessageId` commits the source and target chain ids, the source block hash and number, the nonce, and the optional token and payload legs, including the destination `targetPayloadAddress` and `targetTokenAddress`, but it does not commit the source-chain sender ([MessageId.sol:48-76](#); there is no sender parameter). The account that calls `emitMessage*` is recorded only in the `MessageEmitted` event ([BridgeEvents.sol:70-81](#), the sender field), not in the hashed `messageId` and not in the BLS-attested payload. On the destination, the payload is dispatched as `payloadTargetAddress.call(payload)` with `msg.sender` equal to the executor proxy ([ExecutorLogic.sol:673](#)), so the called contract receives only the source-chosen payload bytes and has no authenticated value telling it which source-chain account authored the message.

Impact: Medium. The bridge provides no way to authorize a message by its source-chain sender: the sender is neither hashed into `messageId` nor surfaced to the payload target, so a destination integration cannot restrict who on the source chain may trigger it. Any account can permissionlessly emit a message to a whitelisted (`target`, `selector`) and have an honest committee deliver it (no relayer manipulation or committee collusion needed, since the message was genuinely emitted), so a target that assumes only a trusted source counterpart drives it is open to impersonation by any source-chain account.

Recommended Mitigation: Bind the authenticated source `msg.sender` into `generateMessageId` so it is committed and BLS-attested, and surface it to the payload target (for example, dispatch the call with the source sender appended or passed as an explicit argument) so a destination app can enforce a source-application allowlist.

Highway: Acknowledged. The bridge currently does not bind the source-chain sender into `messageId` or surface it to the payload target. We agree the real fix is to bind the authenticated sender into the message preimage and pass it to the target, but this is a breaking message-id and dispatch-ABI change requiring lockstep EVM, Substrate, relayer, and integration updates. We are deferring this to a future version. There is no current risk because existing whitelisted payload targets do not authorize behavior based on source sender; until sender binding ships, whitelist review must ensure no target relies on source-sender authentication.

Cyfrin: Rationale accepted with a condition: until the authenticated source sender is bound into the message ID and destination dispatch interface, every whitelist review must reject any target/selector whose authorization or correctness assumes that the call came from a particular source-chain account or application.

7.3 Low Risk

7.3.1 Whitelisting a protocol-internal contract lets `MANAGE_WHITELIST_ROLE` borrow the executor proxy's `MANAGE_ESCROW_ROLE`

Description: The inbound payload-execution path lacks a separation between the *whitelist-management* authority and the *fund-release* authority. A holder of `MANAGE_WHITELIST_ROLE` can whitelist a protocol-internal contract (e.g. Escrow) as a payload target, after which a committee-signed inbound payload message executes that contract's privileged functions with `msg.sender == ExecutorLogicProxy` - an identity that holds `MANAGE_ESCROW_ROLE`. The result is that the whitelist role can be escalated into the escrow-release role and used to drain `RELEASE`-corridor liquidity.

This is a least-privilege / privilege-separation defect, **not** a permissionless drain. It has no honest-operator manifestation: there is no legitimate reason to whitelist Escrow, so the path is only reachable if `MANAGE_WHITELIST_ROLE` is misused or compromised. Because the sole trust-boundary crossing is misuse of that trusted role, severity is Low under a trusted-operator threat model even though the impact (escrow drain) is high. A secondary deploy-script defect (a stale role grant on the hot deployer key) lowers the bar for *who* can supply that whitelist authority, but does not by itself make the path attacker-reachable.

The two contributing defects are described below.

Component 1 - Stale `MANAGE_WHITELIST_ROLE` / `MANAGE_FEE_ROLE` grant on the deployer key (deploy-script vs. runbook mismatch):

In the role-grant block, `Deploy::run` grants `MANAGE_WHITELIST_ROLE` and `MANAGE_FEE_ROLE` to deployer unconditionally, before the single-key / multi-key branch (`script/1_Deploy.s.sol:130-131`). In the multi-key model (`admin != deployer`), the `if` branch renounces only `DEFAULT_ADMIN_ROLE` from the deployer (`script/1_Deploy.s.sol:134-135`); it never revokes `MANAGE_WHITELIST_ROLE` or `MANAGE_FEE_ROLE`. The runbook states the opposite - that after a multi-key deploy "the hot key has no lingering privilege" (`script/README.md:107`). The code therefore contradicts the documentation: the deployer retains live authority over `PayloadWhitelist::registerContract`, `addToWhitelist` (both gated by `onlyRole(MANAGE_WHITELIST_ROLE)` at `src/storage/PayloadWhitelist.sol:85,114`).

Scope-limiting caveat: this is softened by the runbook's own operator flow. In the multi-key model the admin is told to grant the operator roles - including `MANAGE_WHITELIST_ROLE` - to "typically the same hot deployer key" so it can run the whitelist setup scripts (`script/README.md:109,113,115-130`). So a hot key holding `MANAGE_WHITELIST_ROLE` is partly the *intended* steady state, not a pure accident. The defect here is narrowly that (a) the code grants the role in multi-key mode when the documented step-3 flow expects admin to grant it later, and (b) the "no lingering privilege" sentence is inaccurate. It is an independent Low/Informational documentation-vs-code and least-privilege issue.

- Files: `Deploy::run` - `script/1_Deploy.s.sol:130-144`; `script/README.md:107`.

Component 2 - Inbound payload confused deputy borrowing the executor's escrow role (the primary defect):

Inbound execution runs `ExecutorLogic` by `delegatecall` from `ExecutorLogicProxy` (a `TransparentUpgradeableProxy`, `src/proxy/ExecutorLogicProxy.sol:25,82`), so the payload step's low-level call `payloadTargetAddress.call(payload)` in `ExecutorLogic::_validateAndExecutePayload` (`src/logic/ExecutorLogic.sol:659-678`, call at `:673`) executes with `msg.sender == ExecutorLogicProxy`. That proxy was granted `MANAGE_ESCROW_ROLE` (and `MANAGE_STORAGE_ROLE`, `MANAGE_MESSAGES_ROLE`) at deploy time (`script/1_Deploy.s.sol:127-129`). `Escrow::releaseToken`, `releaseNative` are gated by `onlyRole(MANAGE_ESCROW_ROLE)` (`src/storage/Escrow.sol:71,46`), and `onlyRole` resolves the caller as `msg.sender` (`src/access/Authorized.sol:22-23`) - so a call originating from the executor context passes the escrow-role check.

`PayloadWhitelist::validatePayload` (`src/storage/PayloadWhitelist.sol:304-312`) performs only a `whitelist[contractAddress][selector]` membership check; it has no protocol-internal-target exclusion. Consequently, if Escrow is whitelisted for the `releaseToken` / `releaseNative` selectors, the inbound `ExecutorLogic::executeMessagePayload` (`src/logic/ExecutorLogic.sol:315-353`, external, no role gate) drives `Escrow::releaseToken(token, attacker, escrowBalance, msgId)` from an escrow-privileged context.

The core issue is the loss of privilege separation: the system grants `MANAGE_ESCROW_ROLE` only to the executor proxy so that *only validated inbound messages* move funds, and keeps the operational `MANAGE_WHITELIST_ROLE` (a hot key) strictly weaker than fund-movement authority. The confused deputy erases that tiering - anyone holding `MANAGE_WHITELIST_ROLE` can self-escalate to the proxy's `MANAGE_ESCROW_ROLE`.

- Files: `ExecutorLogic::_validateAndExecutePayload` - `src/logic/ExecutorLogic.sol:659-678`;
`PayloadWhitelist::validatePayload` - `src/storage/PayloadWhitelist.sol:304-312`.

Preconditions (why this is Low):

Reaching the drain requires both of the following; only the first is a trust-boundary crossing:

1. **Misuse or compromise of `MANAGE_WHITELIST_ROLE`.** Escrow must be registered and its `release*` selectors whitelisted. An honest operator never does this - there is no legitimate reason to whitelist a protocol-internal contract - so the only trigger is a malicious or compromised holder of that role. Under a threat model where `MANAGE_WHITELIST_ROLE` is trusted and misuse/compromise of that role is excluded from normal attacker reachability, this is Low - a trust-model / key-management failure, not an untrusted-attacker-reachable bug. Absent that assumption, a reviewer could reasonably argue Medium/High, since a weaker hot operator role escalates into full escrow release; the downgrade is a deliberate threat-model choice, not a claim that the impact is small.
2. **A committee-signed inbound payload message** whose `messageId` commits to (`payloadTargetAddress = escrow`, `payload = releaseToken(token, attacker, amount, msgId)`). This is the *normal* inbound payload-message flow, not a separate trusted-failure gate. `executeMessagePayload` verifies a BLS signature over the `messageId` (`src/logic/ExecutorLogic.sol:329`) and reconstructs/validates the `messageId` from the payload and source params (`src/logic/ExecutorLogic.sol:331-343`); the committee attests the `messageId` and the fee service signs a fee quote (`src/logic/EntryLogic.sol:480`) for *every* bridge message, regardless of payload semantics. Once `Escrow.release*` is whitelisted in (1), a malicious user can produce this message through the ordinary permissionless payload path (`EntryLogic::emitMessagePayload`, `src/logic/EntryLogic.sol:457-462`), paying the normal fee, without any committee or fee-service misbehavior. The downgrade therefore rests entirely on the privileged whitelist mutation in (1), not on any assumption that the committee or fee service refuses malicious-but-valid messages.

The original report characterized this path as "permissionless" and requiring "no trusted-operator misconfiguration." That is inaccurate: the draining path is gated on misuse or compromise of `MANAGE_WHITELIST_ROLE`; after that, the remaining steps follow the normal attested inbound payload-message flow. Severity is therefore Low under a trusted-operator / privileged-role threat model.

Impact: If `MANAGE_WHITELIST_ROLE` is misused or compromised, the whitelist role can be escalated into `MANAGE_ESCROW_ROLE`, enabling release of `RELEASE`-corridor escrow balances (ETH, USDT, MOS) to an attacker-chosen address. Impact in isolation is high (full escrow drain), but it requires misuse of a trusted role and so is not exploitable by an untrusted party. The practical value of the finding is least-privilege hardening: without Component 2, compromising the whitelist key alone would only let an attacker route inbound calls to arbitrary *external* contracts (where `msg.sender == proxy` carries no privilege), i.e. limited damage. Component 2 turns a single hot-key compromise (or operator mistake) into total escrow loss by collapsing the intended role separation.

Proof of Concept (privileged precondition):

Setup: a multi-key deployment (`admin != deployer`); `RELEASE` corridors funded (escrow holds ETH / USDT / MOS). Assume the `MANAGE_WHITELIST_ROLE` holder (the hot operator/deployer key) is compromised or acts maliciously.

1. The role holder arms the whitelist: `PayloadWhitelist::registerContract(escrow)` (`src/storage/PayloadWhitelist.sol:83-97`) then `PayloadWhitelist::addToWhitelist(escrow, [releaseToken.selector, releaseNative.selector])` (`src/storage/PayloadWhitelist.sol:112-136`). Both pass `onlyRole(MANAGE_WHITELIST_ROLE)` because `onlyRole` resolves `msg.sender` (`src/access/Authorized.sol:22-23`). This sets `isContractTracked[escrow] = true` and `whitelist[escrow][release*.selector] = true`.
2. A committee-signed inbound payload message is produced with `payloadTargetAddress = escrow` and `payload = abi.encodeWithSelector(releaseToken.selector, token, attacker, amount, msgId)`, amount set to the escrow's balance. The committee BLS-attests the `messageId` only.

3. `ExecutorLogic::executeMessagePayload(messageId, originChainId, sourceBlockHash, sourceBlockNumber, sourceNonce, escrow, payload, proof)` (`src/logic/ExecutorLogic.sol:315-353`) is called. Source chain validates, BLS verifies over `messageId`, the message is marked processed (`:346`), and `_validateAndExecutePayload` runs (`:349`).
4. `_validateAndExecutePayload` extracts the selector (`:667`), `validatePayload(escrow, releaseToken.selector)` returns (pair whitelisted), then `escrow.call(payload)` executes (`:673`). Because `ExecutorLogic` runs by `delegatecall` from `ExecutorLogicProxy`, `msg.sender` inside `Escrow` is the proxy, which holds `MANAGE_ESCROW_ROLE` (`script/1_Deploy.s.sol:128`), so `Escrow::releaseToken's` `onlyRole(MANAGE_ESCROW_ROLE)` check (`src/storage/Escrow.sol:71`) passes.
5. `Escrow::releaseToken` transfers amount of token to attacker (`src/storage/Escrow.sol:79-82`). Repeating with fresh `messageIds` for each `RELEASE`-corridor token (and `releaseNative` for ETH, `src/storage/Escrow.sol:43-58`) empties the escrow.

Note: every step beyond (1) is reachable only because step (1) - whitelisting a protocol-internal contract - succeeded, which requires the trusted role. There is no path to this state without privileged action.

Recommended Mitigation:

1. **Component 2 (primary):** Enforce that protocol-internal contracts can never be inbound payload targets. Either in `_validateAndExecutePayload` reject any `payloadTargetAddress` equal to a known protocol-internal contract (`Escrow`, `MessageStorage`, `Storage`, `FeeCollector`, the proxies, `address(this)`), or have `PayloadWhitelist::registerContract / addToWhitelist` refuse those addresses. Preferably, do not perform payload calls from an identity that carries `MANAGE_ESCROW_ROLE / MANAGE_STORAGE_ROLE / MANAGE_MESSAGES_ROLE` at all - use a dedicated role-less identity for payload calls so a whitelisted target can never borrow the executor's privileges. Also drop `MANAGE_STORAGE_ROLE` from `ExecutorLogicProxy` if the implementation never exercises it. This caps the blast radius of any whitelist-key compromise or operator mistake regardless of the deploy-script defect.
2. **Component 1 (independent):** Confine the `MANAGE_WHITELIST_ROLE / MANAGE_FEE_ROLE` grants (`script/1_Deploy.s.sol:130-131`) to the single-key `else` branch, OR in the multi-key `if` branch explicitly add `authorization.renounceRole(BridgeConstants.MANAGE_WHITELIST_ROLE, deployer);` and `authorization.renounceRole(BridgeConstants.MANAGE_FEE_ROLE, deployer);`, and reconcile the "no lingering privilege" statement in `script/README.md:107` with the operator-grant flow in `script/README.md:115-130` so the documentation matches the resulting role state.

Highway: Fixed in [266e363](#), [7f8253e](#) and [be75d5d](#).

Cyfrin: Verified.

7.3.2 `ExecutorLogic::verifySignature` active-set floor of 128 admits sizes where committee rejection-sampling deterministically exhausts

Description: Before committee selection, `verifySignature` enforces only if (`nActive < BLS12381.COMMITTEE_SIZE`) `revert ActiveSetTooSmall` - that is, `nActive >= 128`. It then performs 128 seats of rejection sampling with `BLS12381.MAX_RETRIES = 32` retries per seat, each drawing `candidate = keccak256(...) % nActive` and rejecting candidates already taken by earlier seats. The BLS12381 library's own NatSpec documents that this is unreliable at the enforced floor:

Rejection sampling becomes unreliable when `nActive` is close to `COMMITTEE_SIZE`. Production deployments should maintain at least `2x COMMITTEE_SIZE` active relayers (256+). With `nActive=128` and 32 retries, the last seat has ~78% failure probability.

The contract enforces a floor of 128, which is exactly the degenerate regime the library warns against; it does not enforce the documented 256+ operational floor. When the operator runs an active set in the `[128, ~2x)` range, committee selection for late seats frequently exhausts all 32 retries and reverts `CommitteeSelectionExhausted`. The safety margin lives only in a code comment, not in a require.

Committee selection is deterministic in (`epochRandomness`, `proof.relayerId`, `proof.slotNumber`). Once a given message's leader-and-slot seed exhausts at any seat, it reverts on every retry - the message is permanently

unexecutable for that (relayerId, slotNumber). A leader has only 5 slots (BLS12381.MAX_SLOTS) and a bounded set of co-relayers to re-attest, so a non-trivial fraction of messages can become permanently undeliverable.

Files:

- ExecutorLogic::verifySignature (src/logic/ExecutorLogic.sol:443-517)
- BLS12381 rejection-sampling reliability note (src/libraries/BLS12381.sol:37-41)

Impact: Availability of the core inbound execution path under a configuration the contract permits. At `nActive` just above 128 (realistic for testnet or early mainnet, where the contract's own floor is the binding constraint), a non-trivial fraction of cross-chain messages deterministically revert in committee selection and cannot be delivered. As with any inbound verification revert, the source-chain leg is already debited; the protocol's design commits that such a revert "leaves source-chain funds debited and pending source-side admin recovery" (per README.md - inbound execution / decimal-precision section), so affected funds are stranded pending out-of-band source-side admin recovery with no destination delivery path.

Proof of Concept: With `nActive` = 128, the 128 committee seats must collectively draw all 128 distinct positions out of a pool of exactly 128. The final seats face a near-saturated pool: by the 128th seat, 127 of 128 positions are already taken, so each of the 32 retries has only a 1/128 chance of drawing the single unseen position. The probability that all 32 retries collide with already-seen positions is approximately 0.78 ($(127/128)^{32}$) - matching the library's documented ~78% last-seat failure estimate.

1. The operator runs an active set with `nActive` at or just above the enforced floor of 128 (the contract accepts this; the 256+ guidance is comment-only).
2. A source-chain message is emitted (debiting the user) and validly signed by a legitimate quorum (`signerCount >= COMMITTEE_THRESHOLD = 87`).
3. A relayer submits `executeMessageToken.verifySignature` passes the `nActive >= 128` check and the BLS-message construction, then enters the 128-seat selection loop. The on-chain reconstruction must reproduce ALL 128 distinct seats before resolving the signer subset.
4. For this (epochRandomness, relayerId, slotNumber) seed, a late seat exhausts all 32 retries; the loop reverts `CommitteeSelectionExhausted(seat, MAX_RETRIES)`.
5. Because selection is deterministic in the seed, every re-submission of the same message reverts identically. The message is permanently undeliverable for that (relayerId, slotNumber), and the source funds remain stranded pending admin recovery.

Textual Step-by-Step Proof:

1. **Initial state.** The protocol operates with an active relayer set whose population is exactly at the contract-enforced floor: `nActive` = 128. `nActive` is derived by `BitmapUtils.buildPrefixSum(activeBitmap)` from the current active-set bitmap fetched via `_activeSetProxy().getCurrentActiveSet()` (src/logic/ExecutorLogic.sol:440-442). The committee parameters are fixed constants: `COMMITTEE_SIZE` = 128 (src/libraries/BLS12381.sol:35), `MAX_RETRIES` = 32 (src/libraries/BLS12381.sol:41), `COMMITTEE_THRESHOLD` = 87 (src/libraries/BLS12381.sol:32), and `MAX_SLOTS` = 5 (src/libraries/BLS12381.sol:44). The only population gate is the floor at src/logic/ExecutorLogic.sol:443-445: `if (nActive < BLS12381.COMMITTEE_SIZE) revert ActiveSetTooSmall(nActive, COMMITTEE_SIZE)` - i.e. any `nActive >= 128` is admitted. The library's own NatSpec at src/libraries/BLS12381.sol:37-41 records that this floor is the degenerate regime: "Rejection sampling becomes unreliable when `nActive` is close to `COMMITTEE_SIZE`. Production deployments should maintain at least 2x `COMMITTEE_SIZE` active relayers (256+). With `nActive`=128 and 32 retries, the last seat has about 78% failure probability." That 256+ operational floor is documentation only; the binding on-chain constraint is `>= 128`.
2. **Setup.** The operator grows the registered active set to exactly 128 relayers (a realistic early-mainnet or testnet condition where the contract's own floor is the binding number). Epoch-based active-set updates flow through `ActiveSetLogic::updateActiveSet`, which validates committee signatures and calls `writeActiveSet` on `ActiveSetStorage` (per the deployment context's active-set update path), producing the bitmap that `getCurrentActiveSet` later returns. With 128 active relayers, the inbound path's floor check at src/logic/ExecutorLogic.sol:443 passes (`128 >= 128`), so the contract treats this active-set

size as fully operational and proceeds to committee reconstruction. No require anywhere enforces the documented $2 * COMMITTEE_SIZE$ reliability bound; the safety margin lives solely in the comment at `src/libraries/BLS12381.sol:38-40`.

3. **Trigger.** A source-chain message has already been emitted on the originating chain, debiting the user (the outbound `emitMessageToken` leg). An off-chain quorum of at least 87 relayers signs it, satisfying `signerCount >= COMMITTEE_THRESHOLD` (`src/logic/ExecutorLogic.sol:408-412`). A relayer submits the normal inbound call `ExecutorLogic::executeMessageToken`, which invokes `verifySignature(messageId, proof)` (`src/logic/ExecutorLogic.sol:382`). All preliminary checks pass: slot bound (`src/logic/ExecutorLogic.sol:384-386`), non-empty signature, pubkey-array bound, non-zero bitmap, threshold, pubkey-count, signature length, and caller-identity (`src/logic/ExecutorLogic.sol:388-437`). The floor check at `src/logic/ExecutorLogic.sol:443-445` passes because `nActive = 128`. Execution computes a deterministic committee seed `committeeSeed = _computeCommitteeSeed(epochRandomness, proof.relayerId, proof.slotNumber)` (`src/logic/ExecutorLogic.sol:462`), then enters the 128-seat reconstruction loop at `src/logic/ExecutorLogic.sol:482`. For each seat the inner loop runs up to `MAX_RETRIES = 32` times (`src/logic/ExecutorLogic.sol:493`), hashing `[committeeSeed | seat_LE | retry_LE]` and computing `candidate = uint256(h) % nActive` (`src/logic/ExecutorLogic.sol:505`). A candidate already taken by an earlier seat is rejected via the seen bitmap (`src/logic/ExecutorLogic.sol:509-513`); an unseen candidate is accepted and the inner loop breaks. The loop reconstructs ALL 128 distinct seats - not just the signer seats - because each seat consumes a distinct position out of the pool of `nActive = 128`. Walk the worst seat: by the 128th seat (`seat = 127`), 127 of the 128 positions are already marked in `seen`, leaving exactly one unseen position. Each of the 32 retries draws `candidate = h % 128`, which lands on the single free position with probability $1/128$ and collides with probability $127/128$. The 32 retries are independent draws keyed on `retry` (`src/logic/ExecutorLogic.sol:498-502`), so the probability that all 32 collide is approximately $0.778 ((127/128)^{32})$.
 4. **Resulting state.** When all 32 retries of the last seat collide - happening for about 78% of distinct `committeeSeed` values at `nActive = 128` - the inner loop hits its final iteration and executes `revert CommitteeSelectionExhausted(seat, MAX_RETRIES)` at `src/logic/ExecutorLogic.sol:514-515` (with `seat = 127, MAX_RETRIES = 32`). `verifySignature` reverts, so `executeMessageToken` reverts. Because committee selection is fully deterministic in (`epochRandomness, proof.relayerId, proof.slotNumber`) (the seed at `src/logic/ExecutorLogic.sol:462` and the per-seat hashing at `src/logic/ExecutorLogic.sol:496-503`), every resubmission of the same message under the same epoch randomness reverts identically. A leader has only `MAX_SLOTS = 5` slots (`src/libraries/BLS12381.sol:44`) and a bounded co-relayer set to re-attempt under different (`relayerId, slotNumber`) seeds, so a non-trivial fraction of messages have no seed permutation that completes 128 distinct seats and become permanently un-executable. The source-chain leg is already committed: per the protocol's documented behavior, an inbound revert "leaves source-chain funds debited and pending source-side admin recovery" (README inbound-execution commitment), so the user's funds are stranded with no destination-side delivery path.
 5. **Impact quantification.** At the configuration the contract itself admits (`nActive = 128`, the exact floor at `src/logic/ExecutorLogic.sol:443`), the last seat alone has an exhaustion probability of approximately $0.778 ((127/128)^{32})$, i.e. roughly 78% of distinct committee seeds (matching the figure documented at `src/libraries/BLS12381.sol:40`). This is a liveness/denial-of-service defect on the core inbound execution path: for a population the contract treats as valid, the large majority of inbound `executeMessage / executeMessageToken` submissions deterministically revert in committee reconstruction and cannot be delivered, while their source legs are already debited. The magnitude is not a transient retry cost - because selection is deterministic per seed, affected messages are permanently undeliverable across resubmissions and across the at most 5 leader slots. Recovery requires growing the active set toward the documented $2 * COMMITTEE_SIZE$ (256+) so the last-seat pool is far from saturated; at `nActive = 256` the worst seat draws from 128 free positions out of 256, collapsing the exhaustion probability to a negligible value. Until then, every fraction of stranded source funds depends on out-of-band source-side admin recovery rather than normal delivery.
- `src/logic/ExecutorLogic.sol:443-445` - the population floor check (`nActive < COMMITTEE_SIZE`) that admits the degenerate `nActive = 128` regime.
 - `src/logic/ExecutorLogic.sol:482-517` - the 128-seat rejection-sampling loop, including `candidate`

= uint256(h) % nActive (:505), the seen-bitmap rejection (:509-513), and the exhaustion revert CommitteeSelectionExhausted(seat, MAX_RETRIES) (:514-515).

- src/logic/ExecutorLogic.sol:462 - the deterministic committeeSeed derived from (epochRandomness, proof.relayerId, proof.slotNumber), which makes the revert reproducible across resubmissions.
- src/libraries/BLS12381.sol:35 (COMMITTEE_SIZE = 128), src/libraries/BLS12381.sol:41 (MAX_RETRIES = 32), src/libraries/BLS12381.sol:32 (COMMITTEE_THRESHOLD = 87), src/libraries/BLS12381.sol:44 (MAX_SLOTS = 5) - the fixed parameters governing the loop.
- src/libraries/BLS12381.sol:37-41 - the NatSpec reliability note documenting the about 78% last-seat failure at nActive = 128 and the comment-only 256+ guidance.

Recommended Mitigation: Raise the code-level floor to match the documented reliability bound: require nActive >= 2 * COMMITTEE_SIZE before sampling. Alternatively, raise MAX_RETRIES to a value that keeps the worst-seat failure probability negligible at nActive == COMMITTEE_SIZE, or replace rejection sampling with a partial Fisher-Yates / swap-based sampler that selects 128 distinct positions in a single pass and never exhausts. The swap-based approach is preferable because it removes the dependence on relayer-count headroom entirely.

Highway: Acknowledged. Exhaustion depends on the headroom between the active set and the 128-seat committee, not on the network's eventual size. Under the independent-draw model used in this finding, the probability that any seat exhausts its 32 retries during one committee reconstruction is approximately 99.2% at 128 active relayers, 2.69e-3 at 160, 7.88e-6 at 192, and 7.92e-10 at 256. We therefore treat 256 active relayers as an operational floor and monitor the active-set size. An ActiveSetTooSmall or committee-exhaustion revert leaves the destination message unconsumed and creates no forged destination state, but source-side funds may remain pending until re-attestation after active-set recovery or administrative source-side recovery. We accept the residual risk on that basis and will raise the code-level floor if monitoring shows the set spending time below 256.

Cyfrin: Rationale accepted with a condition. This acceptance depends on Highway maintaining and monitoring an active set of at least 256 relayers; the EVM contract enforces only 128, so operation below 256 retains the reported inbound-liveness risk.

7.3.3 ExecutorLogic::executeMessage validates the converted amount before the amount>0 guard, making zero-token+payload messages permanently undeliverable

Description: ExecutorLogic::executeMessage (token + payload) runs the converted-amount bound check unconditionally before the transfer block that is gated on if (amount > 0). After decimal conversion, _validateConvertedAmount(amount, chainConfig.minAmount, chainConfig.maxAmount) reverts AmountBelowMinimum whenever amount < minAmount, and minAmount is forced to be at least 1 at corridor-configuration time: Storage::configureTokenBridge, updateTokenBridgeLimits reject minAmount == 0 with InvalidAmountRange.

The presence of the if (amount > 0) guard on the transfer leg shows the function is designed to tolerate a zero token leg alongside a real payload. The converted-amount check at src/logic/ExecutorLogic.sol:224 runs before that guard at line 231, so the tolerant path is unreachable. A message that legitimately commits a payload plus a zero token leg passes source-chain validation (the message-id encoder treats a zero amount as an absent optional value, a valid shape), reconstructs its messageId correctly, and passes BLS verification, then dies on the amount bound on every attempt.

Files:

- ExecutorLogic::executeMessage - src/logic/ExecutorLogic.sol:220-242
- ExecutorLogic::_validateConvertedAmount - src/logic/ExecutorLogic.sol:700-707

Impact: A cross-chain token+payload message carrying a zero token amount is permanently undeliverable via executeMessage. The payload, which may be the entire purpose of the message (a config or governance instruction accompanying a zero-value token leg), never executes. The revert rolls back the markMessageAsProcessed write, so the slot is not consumed, but every retry deterministically hits the same revert. The inbound payload leg has no dedicated admin recovery path: Escrow::recoverNative, recoverToken return escrowed funds but do not deliver a payload. This is a liveness defect, not a fund-theft vector.

Proof of Concept:

1. The source chain emits a token+payload message with `amount = 0`, a registered `tokenId`, and a non-empty payload. The committed `messageId` encodes the amount as an absent optional value.
2. A relayer calls `executeMessage` with `amount = 0`. `_validateSourceChain`, `verifyMessage` (BLS), and `_verifyMessageId` all pass.
3. `markMessageAsProcessed(messageId)` runs at step 4 (`src/logic/ExecutorLogic.sol:213`).
4. `_convertDecimals(0, ...)` returns 0 (the `amount > 0 && converted == 0` dust guard at line 693 does not fire for a zero input).
5. `_validateConvertedAmount(0, minAmount, maxAmount)` reverts `AmountBelowMinimum(0, minAmount)` because `minAmount >= 1`.
6. The whole transaction reverts; the payload at line 242 never executes. Every retry repeats the revert.

Recommended Mitigation: Gate the converted-amount validation, decimal conversion, and inbound-bridge-type check on a nonzero token leg, mirroring the existing `if (amount > 0)` transfer gate. A zero-amount token+payload message then proceeds straight to payload execution. Alternatively, enforce on the source side that any token+payload message carries a nonzero amount.

Highway: Fixed in [951ca98](#), [f74ac14](#) and [aa7a17c](#).

Cyfrin: Verified.

7.3.4 `ActiveSetLogic::setEpochDuration, setRegistrationWindowBlocks` retroactively recompute an effective set's in-flight epoch window

Description: `ActiveSetLogic::setEpochDuration, setRegistrationWindowBlocks` change epoch geometry that the in-progress epoch was already anchored against. Both setters guard only against an outstanding pending set: `_requireNoPendingSet` reverts `PendingSetExists` when the head set's `startBlock` is in the future (`src/logic/ActiveSetLogic.sol:414-418`). They do not block changes while an effective (already-active) set exists with no pending successor.

`_virtualEpochInfo` recomputes the next epoch boundary live from the current parameters: `vStart = refStartBlock + n * epochDuration`, `vEnd = vStart + epochDuration`, `windowStart = vEnd - registrationWindow`, using the current `epochDurationBlocks` and `registrationWindowBlocks` while anchored on `refStartBlock` (the effective set's `startBlock`, frozen at activation) (`src/logic/ActiveSetLogic.sol:485-497`). Changing `epochDuration` mid-epoch recomputes `n` and the `[windowStart, vEnd)` interval for the in-progress epoch against the new geometry even though `startBlock` was chosen under the old geometry. A relayer that timed its deferred submission to fall inside the window under the old geometry can be shifted outside it, or vice versa, and the deferred activation block moves. Because the off-chain (Substrate) side derives the same epoch's boundary from the original geometry independently (that computation is outside this codebase), a mid-epoch EVM-side change also risks desyncing the two sides' view of the in-progress epoch boundary.

The pending-set guard covers only one of the two relevant states; the effective-set-in-progress state is left unguarded. This is the code-level absence of a guard, not a matter of admin care.

Files:

- `ActiveSetLogic::setEpochDuration, setRegistrationWindowBlocks` - `src/logic/ActiveSetLogic.sol:386-419`
- `ActiveSetLogic::_virtualEpochInfo` - `src/logic/ActiveSetLogic.sol:485-497`

Impact: A mid-epoch geometry change desyncs the EVM epoch boundary from off-chain consensus and can move the registration window out from under relayers who timed submissions against the old geometry, causing a deferred active-set update to land outside its expected window. The result is a liveness disruption of the active-set rotation path during honest operation; epoch-geometry tuning and rotation are recurring operational events.

Recommended Mitigation: Extend the guard so neither setter can change epoch geometry while an effective set is in progress with no pending successor (not only when a pending set exists). For example, require that the change take effect only from the next epoch boundary, or block it entirely while any set anchors the current virtual-epoch

computation, so the in-progress `[windowStart, vEnd)` interval is never recomputed under parameters different from those it was anchored against.

Highway: Acknowledged; our earlier rationale was wrong to say there is no real risk. If the parameters change while the registration window is open, an active-set update timed against the previous schedule can revert, and the two sides can temporarily disagree about the in-progress epoch schedule. We still accept this as Low because the rotation is delayed, nothing is lost, and an administrator can re-anchor the schedule immediately. We are not applying the proposed boundary guard because it restricts changes to a narrow point in each epoch and can prevent some order-dependent coupled parameter changes.

Cyfrin: Rationale accepted. The revised response acknowledges the temporary rotation and cross-system schedule-desynchronization risk and accurately records Highway's decision to accept it as Low.

7.3.5 `ExecutorLogic::verifyMessage` BLS proof TTL has no upper bound relative to epoch duration

Description: The only TTL gate on an inbound proof is `if (block.number > proof.ttl) revert BlsProofExpired()` in `verifyMessage` (`src/logic/ExecutorLogic.sol:368`). This is a freshness check - "the proof has not expired" - with no upper bound: there is no `ttl <= block.number + maxTtl` cap and no `ttl < epochEnd` relation (no `MAX_TTL` constant exists). `proof.ttl` is a `uint32` and is bound into the signed BLS message (`messageId || ttl_LE || slotNumber_LE || relayerId_LE, src/logic/ExecutorLogic.sol:424-427`), so it is fixed at signing time and can carry an arbitrarily far-future value.

The TTL gate is independent of committee validity. Inbound verification reconstructs the committee from `getCurrentActiveSet()` at claim time (`src/logic/ExecutorLogic.sol:440`), so once the active set rotates, the recomputed committee (new epoch randomness / membership) no longer matches the signers and `verifySignature` reverts (`PubkeyHashMismatch / BLSVerificationFailed`) - regardless of how much `ttl` remains. Consequently a proof's real usable lifetime is `min(proof.ttl, next active-set change)`, not `[now, ttl]`. Any `ttl` set beyond the next committee change is meaningless: passing the `ttl` check says nothing about whether the proof can still verify, so the on-chain `ttl` advertises a validity window the proof does not actually have.

A message submitted after a rotation but still within `ttl` therefore reverts on the committee mismatch. The revert occurs before `markMessageAsProcessed` (`src/logic/ExecutorLogic.sol:213 / :283`), so the `messageId` is not consumed and remains re-attestable: the current committee can produce a fresh proof over the same `messageId` and a relayer resubmits successfully. The submitter is bound to the proof - `getRelayerIdByOperationalKey(msg.sender) == proof.relayerId` (`src/logic/ExecutorLogic.sol:433-436`) and `proof.relayerId` must be in the current active set (`:447-456`).

Impact: The `ttl` field overstates a proof's validity: it can be set past the next active-set change, where the proof can no longer verify. A message held across an epoch rotation and then submitted within `ttl` reverts on the committee mismatch with the source-chain leg already debited, leaving settlement delayed until the message is re-attested by the current committee. Because the revert precedes `markMessageAsProcessed`, the message is not consumed and recovers via re-attestation rather than admin recovery. The effect depends on inbound verification resolving the committee from the current active set at claim time; this finding's independent content is the missing relationship between `ttl` and the committee-validity window.

Proof of Concept:

1. Block 1000: a message is signed by epoch 5's committee C5, with `ttl = 1500` (well beyond the current epoch). The `ttl` passes the only on-chain bound, which checks just `block.number <= ttl`.
2. Block 1200: the active set rotates; epoch 6 (committee C6) is now in force, so `getCurrentActiveSet()` returns epoch 6's bitmap and epoch randomness.
3. Block 1300: a relayer submits the message - still within `ttl = 1500`. `verifyMessage` passes the TTL check (`1300 <= 1500`), then `verifySignature` re-selects the committee from epoch 6 and checks C5's signature against C6 → mismatch → revert. The source leg was already debited; the message is not marked processed, so it can be re-attested by C6 and resubmitted.

Recommended Mitigation: Bound `proof.ttl` at the verification gate (`src/logic/ExecutorLogic.sol:368`) to the next committee-change boundary rather than to an arbitrary `maxTtl`, so the accepted `ttl` cannot exceed the window in which the proof can actually verify. This is best designed together with pinning committee verification

to the attested epoch: pinning makes a historical-epoch proof verifiable for as long as that epoch is retained, and the `ttl` bound then reflects a real validity window. Because `ttl` is part of the signed `messageId`/BLS preimage recomputed identically on EVM, Substrate, and Solana, any change must land on all chains in lockstep.

Highway: Fixed in [40843cc](#).

Cyfrin: Verified.

7.3.6 `ExecutorLogic::_validateAndExecutePayload` silently consumes a committed zero-target-address payload as a no-op

Description: `ExecutorLogic::_validateAndExecutePayload` only executes the payload when `payload-TargetAddress != address(0) && payload.length > 0` (`src/logic/ExecutorLogic.sol:662`); for a zero target address it returns without reverting and without emitting any event. But the execution gate's treatment of `address(0)` as "no payload" diverges from how the message-id reconstruction treats it. In `executeMessage`, the reconstruction passes `abi.encodePacked(payloadTargetAddress)` into `_verifyMessageId` (`src/logic/ExecutorLogic.sol:205`), where the message-id encoder serializes 20 zero bytes as a present optional value, not an absent one. So a message can commit a non-empty payload together with a zero target address, the reconstructed `messageId` matches, and verification passes.

`markMessageAsProcessed(messageId)` fires at step 4 (`src/logic/ExecutorLogic.sol:213`), before the payload step at line 242. The message slot is therefore permanently consumed, the no-op skip at line 662 drops the committed payload intent silently, and replay protection prevents any retry. The payload never executes and the bridge emits no signal distinguishing "no payload present" from "payload skipped".

The order is the bug: the effect (marking the message processed) is committed before the conditional that may skip the entire payload leg, and the zero-address shape that triggers the skip is a valid committed shape on the message-id side.

Files:

- `ExecutorLogic::executeMessage` - `src/logic/ExecutorLogic.sol:213, 242`
- `ExecutorLogic::_validateAndExecutePayload` - `src/logic/ExecutorLogic.sol:659-678`

Impact: An inbound message that commits a payload with a zero target address has its slot permanently consumed while the payload never executes and no retry is possible. The intended cross-chain action is silently dropped with no event and no recovery path. No fund-creation or double-delivery results; the impact is a silently-dropped payload intent plus a permanently-consumed message slot. The shape requires the source to emit a zero target address with a non-empty payload, a degenerate but not code-forbidden message shape.

Recommended Mitigation: Reconcile the two interpretations of a zero target address so the message-id encoding and the execution gate agree. Either reject a message that commits a non-empty payload with a zero target address before marking it processed, or require the payload target to be present whenever a payload is committed, so a payload-bearing message cannot be silently no-op'd. Do not mark the message processed on a path that then skips the committed payload.

Highway: Fixed in [5d560cb](#).

Cyfrin: Verified.

7.3.7 `Storage, FeeCollector` local pause does not halt bridge flow, leaving the emergency stop inconsistent across the storage contracts

Description: The protocol has two disjoint pause domains. The logic contracts implement `whenNotPaused` as `if (_authorization().paused()) revert ContractPaused()`, so they consult only the global `Authorization` pause. Separately, each storage contract (`Storage`, `MessageStorage`, `FeeCollector`) inherits its own `OpenZeppelin` `Pausable` and exposes its own `pause` / `unpause`. These per-storage flags are never coupled to the global flag, and they gate different surfaces, so the emergency stop behaves inconsistently depending on which contract is paused.

For `Storage` and `FeeCollector`, the per-storage pause does not stop the bridge flow that reads them. `EntryLogic::emitMessage` gates only on `_authorization().paused()`, and the `Storage` functions it calls during a bridge (`getAndUpdateNonce`, `getToken`, `getTokenChainConfig`, `getChainInfo`) are either un-paused or pure views. Notably `getAndUpdateNonce` carries no `whenNotPaused` while every sibling `Storage` configuration setter does, so a `Storage` pause freezes configuration mutation but not nonce issuance on the outbound hot path. `FeeCollector`'s pause likewise applies only to its own configuration setters, not to the read paths the bridge exercises.

`MessageStorage` behaves differently: its `markMessageAsProcessed` (`src/storage/MessageStorage.sol:43`) does carry `whenNotPaused`, and it sits on the inbound execution path (`ExecutorLogic::executeMessage`, `executeMessageToken`, `executeMessagePayload` all call it), so a `MessageStorage` pause does halt inbound execution. The result is an emergency surface where pausing one storage contract halts inbound bridging while pausing another halts nothing the bridge uses, with no single per-storage switch that stops both directions.

Files:

- `EntryLogic::emitMessage` - `src/logic/EntryLogic.sol:358-359`
- `Storage::pause`, `getAndUpdateNonce` - `src/storage/Storage.sol:423-444`
- `FeeCollector::pause` - `src/storage/FeeCollector.sol`

Impact: An operator who calls `pause` on `Storage` or `FeeCollector` during an incident believes they have halted bridge activity, but outbound emission continues; only `Authorization::pause` (or, for the inbound direction specifically, a `MessageStorage` pause) actually stops the corresponding flow. There is no direct fund loss. The harm is degraded incident response and a misleading, inconsistent emergency surface that an operator may rely on under stress.

Recommended Mitigation: Make the pause model coherent: either have the logic contracts' `whenNotPaused` consult the relevant storage contract's pause in addition to `Authorization::paused`, or document explicitly that only `Authorization::pause` halts bridging and rename the per-storage pause functions to reflect that (for `Storage` / `FeeCollector`) they freeze configuration mutation only. Additionally, add `whenNotPaused` to `Storage::getAndUpdateNonce` so a `Storage` pause at minimum stalls nonce issuance, matching its sibling setters.

Highway: Acknowledged; we withdraw our earlier claim that every per-storage pause only freezes configuration. The actual behavior is:

- `Authorization.pause()` is the only full emergency stop for both message directions. It does not freeze storage-layer configuration or stop every active-set and payload-whitelist change.
- `Storage.pause()` gates its own configuration setters, while `getAndUpdateNonce()` remains callable as the outbound hot path.
- `FeeCollector.pause()` gates its applicable configuration setters, while `setFeeSigner()` remains callable. It does not stop message flow.
- `MessageStorage.pause()` is an inbound emergency stop.
- `Escrow.pause()` blocks inbound settlement and recovery. Deposits remain available, so funds can continue entering an escrow that cannot pay out while paused.

The local pauses therefore have different effects. Highway accepts the resulting Low operational-ambiguity risk and uses `Authorization.pause()` as the only full emergency stop for both directions.

Cyfrin: Rationale accepted. The revised response accurately distinguishes `Authorization.pause()` as the only full emergency stop for both directions from `MessageStorage.pause()` as an inbound emergency stop, and records the different effects of the remaining local pauses.

7.3.8 Authorization rotation is impossible on the 4 logic proxies: the documented split-brain mitigation cannot be executed and any rotation permanently desyncs role and pause enforcement

Description: The four transparent proxies (`EntryLogicProxy`, `ExecutorLogicProxy`, `RelayerRegistryProxy`, `ActiveSetProxy`) store their `Authorization` pointer in slot 0, set once in the constructor, with no setter. The proxy

only exposes a `getAuthorization` view. The deployment runbook (out of scope, referenced for context) instructs a batched rotation across all 11 Authorization-dependent contracts, but a call to a proxy's `setAuthorization` routes through the transparent-proxy admin dispatch to the implementation rather than rewriting the proxy's own slot 0, so the proxy's authority cannot be repointed. The seven storage contracts (which inherit `Authorized` and do expose a working `setAuthorization`) can rotate, but the four proxies cannot. A partial rotation that moves only the storage contracts to a new `Authorization` leaves the proxies pointed at the old one: inbound release breaks because `ExecutorLogicProxy` holds `MANAGE_ESCROW_ROLE` on the old authority while `Escrow` now checks the new one, and outbound emission breaks because `EntryLogicProxy` holds `NONCE_MANAGER_ROLE` on the old authority while `Storage` now checks the new one.

Files:

- `EntryLogicProxy` - `src/proxy/EntryLogicProxy.sol:49-64`
- `ExecutorLogicProxy` - `src/proxy/ExecutorLogicProxy.sol:71-84`

Impact: The documented authority-rotation procedure cannot be carried out as written. If an operator attempts it, the partial rotation it produces permanently desyncs role and pause enforcement between the proxies and the storage contracts, bricking both the inbound release path and the outbound emit path with no in-place correction (the proxy slot has no setter). Because the proxies cannot be repointed at all, the only recovery is a full proxy redeployment and re-wiring.

Recommended Mitigation: Add an admin-gated `setAuthorization` to the four proxies that rewrites slot 0 directly (matching the `Authorized` base used by the storage contracts), or document that authority rotation requires re-deploying the proxies and explicitly remove the batched 11-contract rotation from the runbook. Whichever path is chosen, the rotation procedure and the code must agree on a sequence that never leaves the proxies and storage contracts pointed at different `Authorization` instances.

Highway: Fixed in [5293cb9](#).

Cyfrin: Verified.

7.3.9 `ActiveSetLogic::updateActiveSet` enforces only strict epoch monotonicity with no upper bound: a single over-large admin epoch permanently bricks the relayer-driven deferred update path

Description: The only epoch constraint in `updateActiveSet` is `epoch > head.epoch` (if `(epoch <= getActiveSet(...).epoch)` revert `EpochNotMonotonicallyIncreasing()`). There is no upper bound and no tie to block geometry. An admin (or the immediate-activation path) writing an over-large epoch such as `type(uint32).max` makes it impossible for any legitimately-computed future epoch to satisfy the strict-monotonic gate that the `ACTIVE_SET_ADMIN` deferred path depends on, because the deferred path computes its epoch from elapsed block geometry and can never exceed the artificially high head epoch. The decentralized update mechanism is then bricked without any ongoing admin overwrites being required.

Files:

- `ActiveSetLogic::updateActiveSet` - `src/logic/ActiveSetLogic.sol:216-230`

Impact: A single over-large epoch value, written once, permanently disables the relayer-driven deferred active-set update path: no honestly-computed epoch can ever be strictly greater than the inflated head epoch, so every deferred `updateActiveSet` reverts. Harm requires the admin to supply a degenerate value (operator error or a compromised admin key), and the admin-side fault is itself the catastrophic event, so this is Low; the marginal effect of the bug is that the damage is irreversible through the normal update path rather than self-correcting.

Recommended Mitigation: Bound the accepted `epoch` to a sane forward window relative to the current epoch derived from block geometry (for example, reject any `epoch` more than a small constant ahead of the block-derived expected epoch), so a typo or malicious over-large value cannot be written. Tie the monotonic check to the block-derived epoch rather than allowing an unbounded jump.

Highway: Fixed in [35891c5](#).

Cyfrin: Verified.

7.3.10 RelayerRegistryStorage::updateMaxRelayerId is monotonic and never decreases on removal, permanently inflating the required active-set bitmap size after the highest-ID relayer is removed

Description: `updateMaxRelayerId` only ever raises `_maxRelayerId` (if `newId > _maxRelayerId`) `_maxRelayerId = newId`). `removeRelayer` decrements `_totalRelayers` but never recalculates `_maxRelayerId`. Because `ActiveSetLogic` requires every submitted active-set bitmap to be exactly `ceil(maxRelayerId/8)` bytes and committee selection iterates over the full bitmap, once a high-ID relayer (for example id 6000) is registered then removed, `maxRelayerId` stays at that high-water mark forever. Every subsequent `updateActiveSet` carries a bitmap sized to the stale maximum, and the inbound verification path builds a prefix-sum over it, so calldata and gas cost ratchet toward the `MAX_RELAYER_ID` cap of 6000 regardless of how few relayers are actually live.

Files:

- `RelayerRegistryStorage::updateMaxRelayerId` - `src/storage/RelayerRegistryStorage.sol:85-87`

Impact: After the highest-ID relayer is removed, the active-set bitmap can never shrink back to match the live relayer count. The cost of every epoch update and every inbound execution that resolves the active set is permanently inflated (up to a 750-byte bitmap at the 6000 cap) even when only a handful of low-ID relayers remain. This is a permanent cost ratchet rather than a fund-loss path, and the per-operation cost at the cap remains within block gas limits, hence Low.

Recommended Mitigation: On relayer removal, recompute `_maxRelayerId` as the highest id still registered (for example by tracking the live id set or scanning down from the current maximum), so the bitmap size tracks the live relayer population rather than the all-time high-water mark. Keep the monotonic-raise behavior for registration but add a downward reconciliation on removal of the current maximum.

Highway: Acknowledged; Relayer registration is done by trusted service, it reuses "abandoned" IDs. At our expected scale of thousands of active relayers `maxRelayerId` sits nearer cap in normal operation anyway, so bitmap is effectively that size regardless, inflation this describes only matters after a massive teardown to a handful of relayers, which isn't our operating regime and stays well within block gas limits even then.

Cyfrin: Rationale accepted with a condition: the trusted registration service must allocate relayer IDs densely and reuse vacant IDs before assigning higher ones; this policy is not enforced on-chain, and once a high ID is reached the bitmap cannot shrink, although the 6000-ID cap keeps the residual per-call cost bounded.

7.3.11 EntryLogic EIP-712 fee-quote domain separator hardcodes `chainId=1` instead of `block.chainid`, permitting cross-EVM-chain replay if the proxy address and fee signer are reused

Description: `EntryLogic::_domainSeparator` builds the EIP-712 domain with the literal `uint256(1)` as the `chainId` rather than `block.chainid`. The inline comment justifies this on the basis that the per-environment verifying contract (the proxy address) plus a per-environment fee-signer key provide separation. That reasoning holds across dev/staging/prod on a single chain, but it does not defend against the same proxy bytecode deployed at the same address on two EVM chains that share the same fee-signer key, a common pattern for deterministic bridge deployments. A fee quote signed for chain A's proxy recovers to the same `feeSigner` on chain B's proxy because the domain (constant `chainId`, identical verifying contract, identical name and version) is byte-identical. The signed `FeeQuote` struct binds the destination `targetChain` but never the source `localChainId`, so a quote produced for an outbound from chain A is structurally valid as an outbound from chain B, removing the defense-in-depth source binding that an EIP-712-standard `block.chainid` would have provided.

Files:

- `EntryLogic::_domainSeparator`, `_validateFeeQuote` - `src/logic/EntryLogic.sol:163-169, 182-241`

Impact: If the protocol is ever deployed to more than one EVM chain with a deterministic proxy address and a shared fee-signer key, a fee quote signed for one chain can authorize a bridge call on another (for example, a quote signed against a testnet deployment recovering to the same signer on a mainnet deployment that reused the address and key). The harm is bounded by the deployment facts (it requires address and key reuse across chains), and the current configuration does not expose it, hence Low.

Recommended Mitigation: Use `block.chainid` in `_domainSeparator` per the EIP-712 standard so the domain is chain-bound, and add the source `localChainId` to the signed `FeeQuote` fields so a quote is cryptographically

bound to the chain it was produced for. Either change independently closes the cross-chain replay; applying both restores standard chain binding and adds explicit source binding.

Highway: Fixed in [78ac6a6](#), [55896ff](#) and [58aeb46](#).

Cyfrin: Verified.

7.3.12 `RelayerRegistryLogic::removeOperationalKey, addOperationalKey` **change submitter eligibility mid-flight without the active-or-pending guard their sibling carries**

Description: `removeOperationalKey` and `addOperationalKey` lack the `isRelayerActiveOrPending` guard that `removeRelayer` carries. The executor binds a submitting relayer's identity to `msg.sender` via its operational key. If an operational key that a submitter intended to use is removed between the moment a message is assigned to that relayer and the moment it is submitted, the submitter's identity check fails with `RelayerIdentityMismatch`. Unlike relayer removal, there is no guard preventing an operational-key change while the relayer is active or pending, so the key set can shift under an in-flight submission.

Files:

- `RelayerRegistryLogic::addOperationalKey, removeOperationalKey` - `src/logic/RelayerRegistryLogic.sol:276-303`

Impact: A relayer whose operational key is removed mid-flight loses the ability to submit a message it was assigned, getting `RelayerIdentityMismatch`. Recovery is bounded: a relayer's other operational keys remain usable (`removeOperationalKey` reverts `CannotRemoveLastOperationalKey`, so at least one key always survives), so the relayer can resubmit under a surviving key. This is a distinct liveness wrinkle rather than a permanent failure, hence Low.

Recommended Mitigation: Apply the same `isRelayerActiveOrPending` consideration to `addOperationalKey` and `removeOperationalKey` that `removeRelayer` uses, or otherwise sequence operational-key changes so they do not invalidate a key an active relayer may be mid-submission with. At minimum, document that operational-key rotation can transiently fail in-flight submissions and that resubmission under a surviving key is the recovery path.

Highway: Acknowledged; our earlier response addressed the BLS-key finding rather than this operational-key finding. Removal of an operational key is deliberately immediate. If the removed address submits, the call reverts with `RelayerIdentityMismatch` and the transaction rolls back, so the message remains claimable. While the proof remains valid, the same relayer can resubmit it from another operational key because the proof commits to the `relayerId`, not the submitting address, and does not require a new committee attestation. If that relayer does not deliver, another relayer can obtain a new committee attestation and submit under its own `relayerId`. Highway accepts the resulting transient Low liveness risk.

Cyfrin: Rationale accepted. The revised response addresses operational-key removal, the removed sender's fail-closed behavior, and both valid recovery paths.

7.3.13 `ExecutorLogic::_validateSourceChain` **does not reject `originChainId == localChainId` on a mis-configured self-origin corridor, permitting same-chain loopback messages**

Description: `_validateSourceChain` checks only that the origin chain is registered and active (if `(!chainInfo.isActive) revert ChainNotActive(...)`); it never asserts `originChainId != localChainId`. The `messageId` reconstruction encodes both the source and target chain ids, so a self-origin message would carry `sourceChainId == targetChainId == localChainId`. Correctness here relies on the admin not registering the local chain as an active source corridor, rather than on a code-level invariant. A same-chain loopback message that satisfied committee attestation would exercise the `messageId`, decimal-conversion, and release machinery, none of which were designed for loopback, and the shared corridor config keyed by `(tokenId, chainId)` would collapse a hypothetical local-outbound and local-inbound onto the same struct.

Files:

- `ExecutorLogic::_validateSourceChain` - `src/logic/ExecutorLogic.sol:193, 263, 326, 586-589`

Impact: If an admin (by error) registers the local chain id as an active source corridor, the executor would accept self-origin loopback messages instead of rejecting them, exercising code paths not designed for that case. Harm

requires the misconfiguration and a valid committee attestation; the safe state is recoverable by removing the self-origin corridor, hence Low. The defense currently rests on admin configuration discipline rather than a code-level guard.

Recommended Mitigation: Add an explicit `originChainId != localChainId` assertion in `_validateSourceChain`, reverting with a typed error, so a self-origin corridor can never be exercised regardless of registration state.

Highway: Fixed in [1581720](#), [02178a9](#) and [5854e49](#).

Cyfrin: Verified.

7.3.14 `_countBits` (SWAR popcount) returns 0 for `type(uint256).max` instead of 256

Description: `ExecutorLogic._countBits` is a 256-bit SWAR (parallel) popcount. Its accumulation folds the 32 byte-lanes into the low byte, then truncates the result with `& 0xff`:

```
x = x + (x >> 8);
x = x + (x >> 16);
x = x + (x >> 32);
x = x + (x >> 64);
x = x + (x >> 128);
return x & 0xff; // 8 bits → 0..255
```

The popcount of a 256-bit word ranges over `0..256`. The value **256** needs **9 bits** (`0x100`); `& 0xff` can only hold `0..255`. For `x = type(uint256).max` (all 256 bits set) the running total is exactly `0x100`, so the low byte is `0x00` and the carry lands in bit 8, the function returns **0 instead of 256**:

```
_countBits(type(uint256).max) == 0 // should be 256
```

This is the **single** divergent input: every popcount `0..255` is returned exactly (so 255 set bits → 255, but 256 set bits → 0). The boundary is sharp at the all-ones word.

Impact: Low. Dormant under the current parameters, but becomes a live liveness bug if `COMMITTEE_SIZE` is ever raised to 256, a realistic future choice (256 is one full bitmap word).

Today `_countBits` has one caller, `verifyInboundProof` (`ExecutorLogic.sol:409`), gated two lines earlier (step 5, `ExecutorLogic.sol:403-406`):

```
// 5. Validate no bits above COMMITTEE_SIZE are set
if (proof.signerBitmap >> BLS12381.COMMITTEE_SIZE != 0) { // COMMITTEE_SIZE = 128
    revert BridgeErrors.BitmapExceedsCommitteeSize();
}
```

With `COMMITTEE_SIZE = 128` every bitmap reaching the count has bits only in `[0, 128)` → popcount `<= 128`, well inside the `& 0xff` range, so the bug cannot trigger. *But the safety is purely a function of that one constant:*

- **Resize the committee to 256 and the bug goes live.** In Solidity `uint256 >> 256 == 0`, so with `COMMITTEE_SIZE = 256` the step-5 guard degenerates to `0 != 0` (never reverts) and all 256 bits are admissible. A unanimous 256/256 attestation then reaches `_countBits`, which returns 0; `0 < COMMITTEE_THRESHOLD` reverts `InsufficientSigners`.

The defect is a single wrong constant in an otherwise-correct algorithm, so the cost to fix is trivial relative to the latent footgun it removes.

Proof of Concept: Runnable Foundry PoC at [test/poc/L09_CountBitsAllOnes.t.sol](#). It mirrors `_countBits` byte-for-byte (the production function is private) and pins the all-ones divergence against a Brian-Kernighan reference popcount:

```
/// @dev THE BUG: all 256 bits set returns 0, not 256.
function test_countBits_allOnes_returnsZero_DOCUMENTED_BUG() public pure {
    assertEq(_countBits(type(uint256).max), 0, "SWAR returns 0 for all-ones");
    assertEq(_refPopcount(type(uint256).max), 256, "reference returns 256");
}
```

```
}

```

```
[PASS] test_countBits_allOnes_returnsZero_DOCUMENTED_BUG()
```

Run with `forge test --match-path 'test/poc/L09_CountBitsAllOnes.t.sol'` (default osaka evm; no pre-compiles needed). `test_countBits_128bits_correct` is the real caller maximum at `COMMITTEE_SIZE = 128` (why the bug is dormant today); `test_countBits_255bits_correct` confirms the failure is exactly at the all-ones word; the fuzz case asserts equality for every $x \neq \text{type}(\text{uint256}).\text{max}$, **proving** `type(uint256).max` **is the sole divergent input** (no other SWAR drift exists). The same `_countBits` mirror also lives in the combined batch file `test/boundary/SwarAndHistoryEdgeCases.t.sol` alongside the `[[I-07-history-to-buffer-index-no-internal-bounds]]` mirror.

Recommended Mitigation: Delete `_countBits` and call the codebase's already-correct 256-bit popcount, `BitmapUtils.popcount`. That library is already imported and used in this very file ([ExecutorLogic.sol:441](#), `BitmapUtils.buildPrefixSum`), so this removes a redundant second popcount implementation rather than adding a dependency. `BitmapUtils.popcount` does **not** have this bug: it splits the word into four `uint64` lanes, popcounts each with `_popcount64` (which finishes with the $(x * 0x0101\dots01) \gg 56$ multiply-shift trick, a 64-bit max of 64 sits comfortably in the 8-bit top byte), and sums the four results with ordinary `uint256` addition, so the full $0\dots256$ range is held in a `uint256` and never truncated. Verified: `BitmapUtils.popcount(type(uint256).max) == 256`, and it agrees with `_countBits` on every other input.

Separately, **pin** `evm_version` in `foundry.toml` (for example `evm_version = "cancun"` under `[profile.default]`, set to the actual deployment target). It is currently unset, so the compile and test target silently tracks each contributor's installed `forge` (today's default is `osaka`). Pinning it makes this PoC, and the contract's opcode and gas behavior, reproducible and decoupled from local toolchain drift.

Highway: Fixed in [3888013](#).

Cyfrin: Verified.

7.3.15 RelayerRegistryLogic::_verifyPop has no infinity guard

Description: [RelayerRegistryLogic.registerRelayer](#) and [RelayerRegistryLogic.updateBlsKey](#) validate only that `blsPublicKey.length == 128` (128-byte uncompressed EIP-2537 G1); both are `onlyRelayerStorageAdminOrAdmin`. [RelayerRegistryLogic._verifyPop](#) then goes straight to the PoP pairing with **no point-at-infinity check**.

EIP-2537 canonically encodes the G1 point at infinity as 128 zero bytes (and G2 infinity as 256 zero bytes). The PAIRING precompile accepts the identity point, it is in every subgroup, and $e(O, \cdot) = 1$. `_verifyPop` checks $e(\text{blsPublicKey}, H(m)) \cdot e(-G_1, \text{popSignature}) = 1$. The only infinity input that vacuously satisfies this under the real precompile is `blsPublicKey = 0` and `popSignature = 0` together:

$$e(O, H(m)) \cdot e(-G_1, O) = 1 \cdot 1 = 1$$

A non-identity key paired with a zero PoP signature does **not** pass: $e(pk, H(m)) = 1$ would require `pk = 0`, since $H(m) \neq O$. So an all-zero (identity) public key, registered with an all-zero PoP signature, passes Proof-of-Possession.

Impact: Low. Both entry points are `onlyRelayerStorageAdminOrAdmin`, so an identity key can only enter the registry through a privileged action.

The number of identity-key seats is capped at one by the registry's BLS-key deduplication. Every key is indexed by `blsKeyHash = keccak256(blsPublicKey)` and duplicates are rejected on both paths: [registerRelayer](#) reverts `BlsKeyAlreadyRegistered`, and [updateBlsKey](#) does the same on rotation. The EIP-2537 point at infinity has a single canonical 128-byte encoding (all zeros), so it maps to one fixed hash, only one relayer can ever hold it.

If that one identity-keyed relayer is seated in the active set and drawn into a signer slot, it contributes 0 to both the aggregate public key and the aggregate signature; the aggregate still verifies vacuously, so its bitmap bit counts toward the 87-of-128 threshold without a genuine signature, eroding the effective threshold by exactly one seat (**87** → **86**). It does not enable forgery, the other 86 seats still require real signatures. The path is doubly admin-gated

(register the identity key, then seat it) and bounded to a single seat, but the invariant it breaks, *every registered key has a known, exclusively-held secret*, is foundational, so it is worth fixing.

Proof of Concept: [test/poc/L19_InfinityRejection.t.sol](#), two parts.

1. Real-pairing exploit, L19_InfinityRealPairing. The genuine real-world trigger, run against the real EIP-2537 PAIRING precompile (forge default evm_version = osaka). The identity G1 key together with the identity G2 PoP signature pass Proof-of-Possession because $e(0, H(m)) \cdot e(-G1, 0) = 1 \cdot 1 = 1$, so registration succeeds with no infinity guard to stop it:

```
function test_realPairing_identityKeyAndSig_passesPoP_VULN() public {
    bytes memory zeroKey = new bytes(128); // EIP-2537 G1 point at infinity
    bytes memory zeroSig = new bytes(256); // EIP-2537 G2 point at infinity

    vm.prank(relayerAdmin);
    _registry.registerRelayer(1, manager, beneficiary, zeroKey, zeroSig, _opKeys(opKey));

    assertTrue(_registry.isRelayerRegistered(1), "L-19: real EIP-2537 PAIRING accepts identity key +
    ↪ identity PoP");
}
```

2. Aggregate free-seat impact, L19_InfinityAggregateFreeSeat. Once the identity key is seated, it is a free committee slot. Its 0 G1 pubkey is the additive neutral, so the aggregate of {real signer, identity signer} equals the real signer alone, and the 2-member aggregate verifies with the real signer's signature alone, no signature from the identity seat (real BLS vector, sk-signed "highway-msg-1"):

```
function test_identitySigner_isFreeSeatInAggregate_VULN() public view {
    // Baseline: the real signature verifies against the real key.
    assertTrue(BLS12381.verifyAggregateSignature(SIG_REAL, MSG, PK_REAL), "baseline: real sig
    ↪ verifies");

    // A 2-member committee: one real signer + one identity (G1 point at infinity) signer.
    bytes memory identity = new bytes(128); // 128 zero bytes = identity G1
    bytes[] memory committee = new bytes[](2);
    committee[0] = PK_REAL;
    committee[1] = identity;

    // Aggregating the identity changes nothing: agg == PK_REAL (additive neutral).
    bytes memory agg = BLS12381.aggregatePubkeys(committee);
    assertEq(keccak256(agg), keccak256(PK_REAL), "identity is the additive neutral in the aggregate
    ↪ pubkey");

    // VULN: the aggregate verifies with ONLY the real signer's signature.
    assertTrue(
        BLS12381.verifyAggregateSignature(SIG_REAL, MSG, agg),
        "L-19: {real + identity} aggregate verifies with the real signature alone"
    );
}
```

Together the two parts cover the full chain against the real precompiles: part 1 shows the identity key gets *in* (PoP accepts an all-zero key + all-zero PoP, so it registers), and part 2 shows *why it matters* (a seated identity signer satisfies its committee bit for free, so the 87-of-128 quorum is met with one fewer genuine signature).

Recommended Mitigation: Reject the point at infinity in `_verifyPop`. The delivered tree has **no** infinity handling, `_verifyPop` goes straight to the pairing, so the fix is to **add** the three pieces below:

1. A `BLS12381.isInfinity(bytes)` helper (memory-safe assembly, word-wise OR with a masked tail; works for any length), to be added to `src/libraries/BLS12381.sol`.
2. `BridgeErrors.BlsKeyIsInfinity` and `BridgeErrors.PopSignatureIsInfinity` typed errors, to be added to `src/libraries/BridgeErrors.sol`.
3. Two guards at the top of `_verifyPop` (before the pairing call):

```

if (BLS12381.isInfinity(blsPublicKey)) revert BridgeErrors.BlsKeyIsInfinity();
if (BLS12381.isInfinity(popSignature)) revert BridgeErrors.PopSignatureIsInfinity();

```

Centralizing the check in `_verifyPop` guards both `registerRelayer` and `updateBlsKey` in one place, and it runs before the (expensive) pairing precompile, so it is gas-cheap on the unhappy path.

Highway: Fixed in [1ac4a98](#), [470dbb7](#) and [4a1f5ba](#).

Cyfrin: Verified.

7.3.16 MessageId cannot encode Some(0) or Some(empty)

Description: The protocol spec ([Message ID formula](#)) specifies the five optional `messageId` fields as genuine SCALE Options:

None is encoded as `0x00`; `Some(x)` as `0x01 || x`. This SCALE Option encoding is used on both chains to ensure byte-level parity.

That is **presence-based**: `Some(0)` and `Some(empty)` are valid, distinct from `None`. The EVM `MessageId.generateMessageId` does **not** encode by presence, it substitutes a **sentinel**, and inconsistently so:

- Numeric fields key `None` off the value. `_scaleOptionU128(amount)` and `_scaleOptionU32(tokenId)` return `0x00` (`None`) when `value == 0`, with **no** `0x01 || 0...` **branch**, so `Some(0)` is unrepresentable.
- Bytes fields key `None` off the length. `_scaleOptionBytes/ _scaleOptionAddress` return `0x00` (`None`) when `length == 0`, so `Some(empty)` is unrepresentable, while a payload whose *content* is zero (e.g. `0x00000000`, length 4) is correctly `Some(0x01 || compact(4) || 00000000)`.

So the encoder applies two different `None` rules (value-0 for numerics, length-0 for bytes) and neither matches the spec's true `Option`. The bytes path actually *does* presence-based encoding for non-empty content, which makes the numeric value-collapse the clear outlier.

Impact: Low, and the practical trigger is latent, but it is a **spec-conformance defect** with a concrete consequence:

- The EVM cannot produce or reconstruct a `messageId` whose preimage contains `Some(0)` (amount/tokenId) or `Some(empty)` (payload/address). Any spec-conformant counterpart that emits such a field produces a `messageId` the EVM reconstructs differently (its sentinel maps the field to `None`), so `_verifyMessageId` reverts `InvalidMessageId`, a cross-chain liveness DoS.
- It is latent today because `Some(0)/Some(empty)` are economically meaningless and partly prohibited (`tokenId 0` reserved, `amount > 0` required, `minAmount >= 1`), so neither chain's normal flow emits them. It is realized only by a spec-following peer emitting one, or a future change that treats `0/empty` as a meaningful `Some`, e.g. storage already carries dormant `tokenId = 0` support (`isTokenRegisteredFlag/ _isTokenRegistered`, `Storage.sol:46-47/:602-603`) held off only by `registerToken's tokenId != 0` guard (:120), and relaxing that lone guard would make a real `token-0` transfer encode as `None` here.

Proof of Concept: Runnable Foundry PoC at [test/poc/L17_OptionSentinel.t.sol](#) (passes), exercising the real `MessageId.generateMessageId`. It computes the EVM `messageId` for a message with `tokenId = 0` (empty payload/addresses, zero amount), reconstructs the spec preimage two ways, with `tokenId` as `None` (`0x00`) and as `Some(0)` (`0x01 || u32le(0)`), and asserts the EVM output equals the `None` reconstruction (so the encoder maps `0 -> None`) and differs from the `Some(0)` reconstruction (so the EVM cannot produce or reconstruct a `Some(0)` field). A spec-conformant peer emitting `Some(0)` therefore hashes to an id the EVM rebuilds as `None`, and `_verifyMessageId` reverts `InvalidMessageId`.

```

function test_L17_evmCollapsesZeroToNone_andCannotRepresentSome0() public {
    bytes32 evmId = MessageId.generateMessageId(SRC, TGT, BH, BN, NONCE, "", "", 0, "", 0);
    assertEq(evmId, keccak256(_preimage(false)), "EVM encodes tokenId 0 as None (0x00 sentinel)");
    assertTrue(evmId != keccak256(_preimage(true)), "Some(0) is unrepresentable on EVM (sentinel !=
    ↪ presence)");
}

```

Recommended Mitigation: There are two ways to align it:

1. **Canonicalize the sentinel.** Amend the spec to state that an amount/tokenId of 0 and an empty payload/address encode as None, and Some(0)/Some(empty) are not valid messages. The EVM is then conformant as-is, and the peer (Substrate) must reject/normalize Some(0)/Some(empty) to match. At minimum, make the two EVM sentinel rules (value-0 for numerics, length-0 for bytes) consistent and documented.
2. **Honor true presence, the way SCALE encodes Option<T>.** SCALE uses a 1-byte presence discriminant that is independent of the value: None is 0x00, Some(x) is 0x01 || encode(x), so Some(0) (0x01 || 0...) and Some(empty) (0x01 || 0x00) are distinct from None by construction. The Substrate side already does exactly this (it encodes real Option<T> via .encode()).

This shares its root, MessageId's non-spec Option encoding, with the non-injective-encoding High issue: that issue is the address-framing collision facet, this is the unrepresentable-Some facet. A presence-based, self-delimiting re-encoding (Option 2) addresses both.

Highway: Fixed in [92cddcd](#) and [923bdeb](#).

Cyfrin: Verified.

7.3.17 Outbound payload cap is not per-destination.

Description: EntryLogic bounds the outbound payload only by a single global constant, `MAX_PAYLOAD_SIZE = 4096`, checked at emit and reverting `PayloadTooLarge` above it ([EntryLogic.sol:391-392](#) and [:469-470](#)). The same cap applies to every `targetChain`, and the registered per-chain config carries no payload limit. Some target chains cannot carry a 4096-byte payload: per the Solana documentation, the maximum size of a Solana transaction is 1,232 bytes (<https://solana.com/docs/core/transactions>), so a 4096-byte payload cannot be delivered to a Solana destination in a single transaction.

Impact: Low. A user can submit, and pay the fee for, an outbound message whose payload is within the 4096 cap but above what the destination can receive; the emit succeeds and the message is committee-attested, yet it can never be delivered on that leg, and any token leg's escrowed value is stranded with it pending admin intervention. This is a liveness/availability gap, not a theft path, and is bounded by how often large payloads target a low-capacity chain, hence Low.

Recommended Mitigation: Make the cap per-destination: add a `maxPayloadSize` to each target chain's registered config in `Storage`, and validate the payload against the target chain's limit in `EntryLogic`, reverting before fee collection.

Highway: Acknowledged. The per-destination payload-size limit is enforced off-chain by the fee service, which gates every emit, sees the full message, and knows each destination's capacity, so an oversized payload is rejected before the user pays.

Cyfrin: Rationale accepted with a condition: every live EVM emit must require a nonzero fee signer, and the fee service must reject the complete message against the destination adapter's current transaction-envelope limit - including Solana's current 700-byte cap - before signing; neither condition is enforced by the EVM contracts.

7.3.18 No rate limit, withdrawal delay, or circuit breaker bounds inbound damage

Description: The only damage control on inbound delivery is the per-corridor `[minAmount, maxAmount]` cap ([ExecutorLogic.sol:700](#)). There is no per-period or rolling volume cap, no withdrawal delay or challenge window (delivery is final within one transaction), and no circuit breaker. Pause exists but is reactive and admin-gated ([Escrow.sol:164](#)), so it cannot front-run an attacker-timed transaction.

Impact: Low, defense-in-depth. While the honest 87-of-128 committee assumption holds there is no exploit. But nothing bounds the damage if that assumption breaks (collusion) or a verification bug lets one bad message through: a single block can drain a RELEASE corridor's full escrow and mint unbounded on a MINT corridor, with no automatic backstop. This concerns the inbound path itself, separate from the admin-key centralization note.

Recommended Mitigation: Add a preventive bound behind committee verification: a per-corridor rolling-window volume cap and/or a withdrawal delay above a configurable amount (so large releases mature with a cancel

window), plus a low-trust guardian pauser separate from `DEFAULT_ADMIN_ROLE`. This is defense-in-depth hardening that goes beyond what the protocol strictly requires.

Highway: Acknowledged. Defense-in-depth; gated onboarding (NFT + registrar) keeps the committee honest-majority and a malicious 86-of-128 quorum negligible, so the inbound bounds are post-launch hardening, with reactive pause, per-corridor `[min, max]`, and a possible low-trust guardian pauser retained.

Cyfrin: Rationale accepted with a condition: every EVM active set must be restricted to registrar-vetted, NFT-backed, independently controlled operators with a strict-sub-third malicious share under the current 86-of-128 quorum; current EVM `master` enforces neither NFT backing nor that composition, and a quorum compromise or verifier bypass still has no aggregate inbound-loss backstop.

7.4 Informational

7.4.1 `Storage::_validateBridgeConfiguration` does not constrain per-corridor escrow to a single validated protocol escrow

Description: `bridge.escrow` is a per-corridor field set per `(tokenId, sourceChainId)` through `Storage::configureTokenBridge` (`onlyRole(MANAGE_STORAGE_ROLE)`, `src/storage/Storage.sol:172-224`). The only validation it receives is in `_validateBridgeConfiguration` (`src/storage/Storage.sol:627-645`), which (a) rejects role-mismatched bridge pairs - (ESCROW, MINT) and (BURN, RELEASE) revert `InvalidBridgePair` - and (b) requires the escrow address to be non-zero exactly when the corridor needs one (an ESCROW outbound or RELEASE inbound), reverting `InvalidEscrowConfiguration` otherwise (:640-644). The function is internal pure, so it performs no on-chain checks: there is no `code.length` check that the address is a contract, and no registry/role/interface check that it is a genuine protocol escrow.

On an inbound RELEASE, `ExecutorLogic` passes the corridor's `bridge.escrow` to `_handleRelease`, which calls `IEscrow(escrowAddress).releaseToken / releaseNative` on it. A non-zero externally-owned account (or any non-escrow address) passes configuration but has no matching code, so every release on that corridor reverts, and any outbound ESCROW lock routed to that address is stranded with no way to release it.

The escrow field is intentionally per-corridor: a token's corridors may legitimately point at the same escrow (pooled liquidity) or at distinct escrows (per-corridor isolation). Distinct escrows are a valid deployment choice - pooling all corridors into one escrow concentrates collateral and is a cross-chain single point of failure - so the per-corridor escrow relationship is by design, not a defect, and the contract should not force a single escrow per token.

Impact: A corridor whose `bridge.escrow` is set to a non-contract or non-protocol address passes configuration but cannot service releases: inbound releases revert and any outbound lock routed to the address is stranded. This is an admin-only (`MANAGE_STORAGE_ROLE`) misconfiguration that fails on the first lock/release, so it surfaces at corridor setup rather than after a corridor is in production use. There is no externally reachable path and no funds at risk beyond the self-inflicted, immediately-visible broken corridor.

Recommended Mitigation: In `_validateBridgeConfiguration` (promoting it from pure to view), require the escrow address to have nonzero `code.length` so an EOA cannot be configured as a corridor escrow. Optionally, add a stronger authenticity check that the address is a genuine protocol escrow - for example an on-chain allowlist of deployed escrows, or a role/interface marker the escrow carries. Do not enforce that all corridors for a token share one escrow: per-corridor escrows are a legitimate design, and a single shared escrow concentrates collateral into a cross-chain single point of failure.

Highway: Fixed in [dd06eab](#).

Cyfrin: Verified.

7.4.2 `ExecutorLogic::_handleMint` does not reject a zero recipient for a MINT token that permits zero-address mints, burning bridged tokens to `address(0)` while the message is consumed

Description: On inbound execution of a MINT-type corridor, `ExecutorLogic::_handleMint` mints the bridged amount to the message-supplied recipient. The function guards only against a native token address (if `(tokenAddress == address(0)) revert CannotMintNative()`); it never checks that `recipient != address(0)`. The recipient is `tokenTargetAddress`, which is bound by the committee-attested `msg.sender`, so a relayer cannot forge it, but a source-side message that legitimately encodes `tokenTargetAddress = address(0)` will pass `msg.sender` verification, mark the message as processed, and then mint to the dead address. The post-mint balance-delta assertion (`minted == amount`) is checked against `balanceOf(recipient)` for that same zero recipient, so for any MINT token whose mint implementation permits crediting `address(0)`, the delta check passes and the mint succeeds. This is asymmetric with the release path: `Escrow::releaseNative`, `releaseToken` reject a zero `receiveAddress`, and the inbound release helper rejects a zero escrow address.

Files:

- `ExecutorLogic::_handleMint` - `src/logic/ExecutorLogic.sol:731-746`

Impact: For a MINT corridor whose token contract permits minting to `address(0)`, an inbound message carrying a zero recipient consumes the `msg.sender` (it is marked processed and `TokenMinted` is emitted) while the bridged amount is minted to the unrecoverable dead address. The source-chain leg has already been debited, so the value

is permanently stranded with no inbound re-delivery path. The impact is bounded to a degenerate message shape (the source must encode a zero target) and to MINT tokens that do not themselves reject zero-address mints, hence Low.

Recommended Mitigation: Add an explicit `recipient != address(0)` check at the top of `_handleMint`, reverting with a typed error, mirroring the zero-address rejection already present on the release path. This keeps every other branch the function handles (valid recipient, non-conforming token rejected via `MintAmountMismatch` / `MintBalanceDecreased`) intact.

Highway: Fixed in [144d80c](#).

Cyfrin: Verified.

7.4.3 `EntryLogic::emitMessage`, `emitMessageToken`, `emitMessagePayload` do not validate destination-address byte-width against the target chain format, stranding burned or escrowed funds

Description: The outbound entry functions accept `tokenTargetAddress` and `payloadTargetAddress` as raw bytes `calldata` and perform no length or format validation before debiting the user (burn or escrow) and emitting the cross-chain message. `MessageId::generateMessageId` encodes a 32-byte value as the Substrate `Option<AccountId32>` form but any other non-empty length as a variable-width byte vector. The destination peer (Mosaic, a Substrate runtime registered as the cross-chain peer) reconstructs and matches the `messageId` using its native 32-byte account representation of the recipient. If a user supplies a non-32-byte target address (for example a 20-byte EVM-style address), the EVM produces a `messageId` computed over the variable-width encoding while the destination, expecting a 32-byte account, either cannot reconstruct a matching `messageId` or resolves a different recipient. The funds are already burned (BURN corridor) or locked in `Escrow` (ESCROW corridor) at emit time, but the destination cannot credit the intended recipient. There is no on-chain send-side guard that the address width matches the destination format, and the protocol explicitly supports a no-fee-signer mode in which fee-quote validation returns immediately, removing even the off-chain backend as a validation point.

Files:

- `EntryLogic::emitMessage`, `emitMessageToken`, `emitMessagePayload` - `src/logic/EntryLogic.sol:375-448, 457-515, 525-593`
- `MessageId::generateMessageId` - `src/libraries/MessageId.sol:95-103`

Impact: For BURN corridors the user's tokens are destroyed on the EVM side with no destination credit and no on-chain recovery path (burn-and-mint holds nothing to recover; recovery would require a source-side admin re-mint). For ESCROW corridors the funds sit in `Escrow` but the intended recipient is never credited on Mosaic, and recovery requires an admin `Escrow::recoverToken` / `recoverNative` call. The harm manifests at zero protocol cost the moment any caller emits a wrong-width address while the fee signer is unset, and even with a fee signer set the only thing standing between the user and burned funds is an undocumented off-chain format check. Bounded to the caller's own funds with no third-party victim class, hence Low.

Recommended Mitigation: Add a per-target-chain expected-address-width (or format-class) field to the registered chain configuration and validate `tokenTargetAddress.length` / `payloadTargetAddress.length` against it in each emit function before debiting funds. For the Substrate/Mosaic peer, require exactly 32 bytes. Reject mismatches with a typed error.

Highway: Acknowledged; Loss is self-inflicted, caller supplies a malformed-width address for their own funds, with no third-party victim.

Cyfrin: Rationale accepted with a condition: whenever EVM `EntryLogic` is live, its fee signer must remain nonzero and the signer must reject noncanonical destination addresses - including non-32-byte Mosaic targets - before issuing a quote; the EVM contracts do not enforce destination-address width themselves.

7.4.4 `ActiveSetLogic::updateActiveSet` **re-reads** `maxRelayerId` **at submission time: a concurrent** `registerRelayer` **forces a** `BitmapTooSmall` **revert on the deferred update path**

Description: `updateActiveSet` re-reads `maxRelayerId` from `RelayerRegistryStorage` at submission time and enforces that the submitted `activeBitmap` is exactly `ceil(maxRelayerId/8)` bytes (`_validateBitmap` rejects both too-small and too-large). The off-chain `ACTIVE_SET_ADMIN` role that builds and signs the bitmap must size it against a `maxRelayerId` snapshot taken earlier. Because `registerRelayer` monotonically increases `maxRelayerId` (via `RelayerRegistryStorage::updateMaxRelayerId`), a relayer registered between the moment the admin reads `maxRelayerId` and the moment its `updateActiveSet` transaction is mined makes the live `maxRelayerId` larger than the bitmap was sized for, and the submission reverts `BitmapTooSmall`. The in-code comment acknowledges this and instructs callers to re-read `maxRelayerId` immediately before signing, confirming the staleness is real and the mitigation is operational rather than code-level.

Files:

- `ActiveSetLogic::updateActiveSet` - `src/logic/ActiveSetLogic.sol:142-147, 543-562`

Impact: Liveness/DoS of the deferred active-set update path during concurrent relayer registration. Bounded to a clean revert with no fund loss and no incorrect state written; the relayer recovers by re-sizing against the fresh `maxRelayerId`. Because the registration window is finite (roughly 100 blocks at the configured value), repeated registrations can cause the window to be missed entirely for that epoch.

Recommended Mitigation: Accept a bitmap sized to at least `ceil(maxRelayerId/8)` and require only that the trailing bits beyond `maxRelayerId` are zero (relax the exact-length upper bound to a lower-bound plus zero-padding check), or have the off-chain signer commit to the `maxRelayerId` it sized against and validate the bitmap against that committed value rather than the live one. Either removes the race without weakening the trailing-bit guarantee.

Highway: Acknowledged; Race is real but narrow and self-recovering. `BitmapTooSmall` can only fire when a concurrent `registerRelayer` pushes `maxRelayerId` across a byte boundary (every 8th registration) and lands in signing→mining window of an `updateActiveSet` tx. Result is a clean revert, no state written, no funds at risk, and the submitter recovers by re-reading fresh `maxRelayerId` and resubmitting, which in-code guidance already directs.

Cyfrin: Rationale accepted: this is an atomic, recoverable `BitmapTooSmall` liveness edge, and the updater can rebuild against the latest `maxRelayerId` and resubmit; the trigger is any registration that increases the required bitmap byte length, not necessarily every eighth registration.

7.4.5 `5_SetupFeeCollection` **setup script is orphaned from every deployment path: the bridge goes live with fee validation disabled and zero fees collected**

Description: `5_SetupFeeCollection.s.sol` is the only script that calls `FeeCollector::setFeeSigner / setExecutionRecipient / setPlatformRecipient`, and it is invoked by no deployment path. The all-in-one `Setup.s.sol` wrapper runs `Deploy`, `DeployTokens`, `ConfigureBridge`, `FundEscrow`, `ConfigureWhitelist`, and `RegisterRelayers` and never runs the fee step; both the deployment runbook (`script/README.md`) and the Sepolia deployment guide list the setup scripts without it. After any documented deployment, `FeeCollector._feeSigner == address(0)`, and `EntryLogic::_validateFeeQuote` returns early when the fee signer is the zero address. Every `emitMessage / emitMessageToken / emitMessagePayload` therefore skips fee-quote signature and TTL validation entirely and collects no protocol fee. The runbook compounds this with a numbering collision: its "step 5 is skipped" note refers to the `revoke-operator-roles` step, not the fee script, which is never referenced anywhere in either runbook.

Files:

- `SetupFeeCollection::run` - `script/5_SetupFeeCollection.s.sol:33-83`
- `SetupHighway::run` - `script/Setup.s.sol:28-54`

Impact: Every documented deployment produces a bridge that silently collects zero protocol/execution fees and performs no fee-quote validation, for the lifetime of the deployment until an operator independently discovers the orphaned script and runs it. User funds are not directly at risk (the disabled-fee path is a documented runtime mode), but the entire fee-revenue mechanism the protocol built is inert by default and a user can supply a zero-fee or any unsigned quote and pay nothing.

Recommended Mitigation: Add `5_SetupFeeCollection` to the documented production runbook (`script/README.md` and `script/SEPOLIA_DEPLOYMENT.md`) as an explicit required step before announcing the bridge live, and resolve the "step 5" numbering collision so it unambiguously refers to either the fee script or the revoke step. At minimum, document that a deployment with the fee signer left at `address(0)` collects no fees, so the operator makes an informed choice.

Highway: Fixed in [720f5d2](#).

Cyfrin: Verified.

7.4.6 Deployment scripts silently fall back to the publicly-known Anvil account-0 private key when `PRIVATE_KEY` is unset, with no chain-id guard

Description: Every deployment script resolves its broadcast key using `vm.envOr("PRIVATE_KEY", ...)`, where the fallback is the well-known Anvil/Hardhat account-0 private key. When `PRIVATE_KEY` is unset, the script does not revert; it silently broadcasts from that publicly known development key. In `1_Deploy.s.sol`, that key becomes the deployer and (in single-key mode) receives `DEFAULT_ADMIN_ROLE` plus all operator roles. The fallback is documented as intentional for local Anvil, but there is no `block.chainid` guard anywhere in the scripts (zero `require(block.chainid == ...)` occurrences), so the same publicly-known key is the silent default even when the RPC endpoint points at a public testnet or mainnet.

Files:

- `Deploy::run - script/1_Deploy.s.sol:53-54`

Impact: A forgotten environment variable on a public-network broadcast deploys the entire bridge under a private key known to the public, handing `DEFAULT_ADMIN_ROLE` (and in single-key mode all operator roles) to an attacker-controllable address. Exploitation requires operator error and a funded public-key address, so this is Low, but the silent fallback (versus a hard revert) materially raises the footgun.

Recommended Mitigation: Prefer Foundry's encrypted keystore (`--account <name>`) and drop the env-var key entirely. If the env fallback is retained for local dev, gate it: `require(block.chainid == 31337 || vm.envExists("PRIVATE_KEY"), "set PRIVATE_KEY for non-local chains")` so a non-Anvil broadcast with an unset key reverts instead of silently using the public account-0 key. Add a `require(block.chainid == <expected>)` guard for testnet/mainnet runs.

Highway: Fixed in [a8fe2cd](#).

Cyfrin: Verified.

7.4.7 Deployment scripts hardcode Highway `LOCAL_CHAIN_ID=3` and Mosaic peer `chainId=2` with no `block.chainid` assertion: a misdirected broadcast deploys a mis-keyed bridge

Description: `EntryLogic` and `ExecutorLogic` are constructed with `BridgeConstants.LOCAL_CHAIN_ID (=3)`, and `ConfigureBridge` registers the Mosaic peer as `chainId 2` by default. These Highway-internal chain ids must match what the paired Substrate runtime expects. No script asserts that `block.chainid` equals an expected EVM network (zero `require(block.chainid == ...)` across all scripts), and deployment artifacts are written under a path derived from `block.chainid` with no expected-value check. A misdirected RPC endpoint would therefore deploy a fully-wired bridge keyed to the wrong EVM network without any guard tripping. Because `localChainId` is set as an immutable in the logic constructors, a wrong-network deploy is uncorrectable in place.

Files:

- `Deploy::run - script/1_Deploy.s.sol:78-79`
- `ConfigureBridge::run - script/2_ConfigureBridge.s.sol:27, 49`

Impact: A wrong RPC endpoint on broadcast deploys a bridge whose immutable local chain id and registered peer chain id are mis-keyed relative to the network it landed on, with no on-chain guard to catch it. Harm requires operator error (wrong RPC), and the on-chain artifact is still internally consistent, but the immutability of `localChainId` means correction requires a full redeploy rather than a configuration fix, hence Low.

Recommended Mitigation: Add a `require(block.chainid == <expected>)` assertion in `1_Deploy.s.sol` (and the configuration scripts) gating each network's deploy against the EVM chain id it is meant for, so a misdirected broadcast reverts before any contract is created. Expose the expected chain id as a script parameter or environment variable checked against `block.chainid`.

Highway: Fixed in [6fc4005](#).

Cyfrin: Verified.

7.4.8 `EntryLogic MessageEmitted` omits `sourceBlockHash`, one of the committed inputs to the consensus-critical `messageld`

Description: The `messageld` is generated from `blockhash(block.number - 1)` (the source block hash), `uint64(block.number)`, the nonce, and the address/payload/token/amount fields. The `MessageEmitted` event emits `messageId`, `block.number` (the current block), and nonce, but not the source block hash. On the inbound side, the relayer must supply that source block hash to the executor, where `messageld` reconstruction checks it. Because the relayer is handed the `messageId` directly in the event and can independently fetch `blockhash(blockNumber - 1)` from a source-chain archive node, the source block hash is deterministically recoverable, so this is not a correctness break. It does force every relayer to perform an extra parent-block-hash lookup that a complete event would have avoided, and it leaves one of the committed `messageld` inputs absent from the canonical emission.

Files:

- `EntryLogic::emitMessage, emitMessageToken, emitMessagePayload` - `src/logic/EntryLogic.sol:418-447`

Impact: Off-chain consumers must perform an additional archive-node lookup to reconstruct the `messageld` preimage, because the canonical event omits a committed input. No funds at risk and no consensus break (the value is recoverable on-chain), but the emission is incomplete relative to the data the `messageld` commits to.

Recommended Mitigation: Add the source block hash (`blockhash(block.number - 1)`) to the `MessageEmitted` event fields so the full `messageld` preimage is available directly from the event stream without an extra archive-node query.

Highway: Acknowledged; The relayer already obtains the parent block hash from batched block-header reads performed alongside `eth_getLogs`, so adding `sourceBlockHash` to `MessageEmitted` would trade an already-paid off-chain lookup for permanent per-message event gas and cross-repo decoder changes.

Cyfrin: Rationale accepted: the committed parent hash is deterministically recoverable from the emitting block's header, so omitting it creates an off-chain lookup tradeoff rather than a correctness or funds risk, while adding it would impose permanent event gas and decoder changes.

7.4.9 `ActiveSetLogic ActiveSetUpdated` emits only `activeCount`, not the full `activeBitmap` or `maxRelayerId` that defines committee membership

Description: `updateActiveSet` writes a full active relayer set (`epoch`, `startBlock`, `activeBitmap`, `maxRelayerId`, `epochRandomness`) to storage, but `ActiveSetUpdated` emits only (`epoch`, `startBlock`, `activeCount`, `epochRandomness`). The `activeBitmap` is the data that defines which relayers are active and is consumed on-chain for committee selection; the event reports only the population count of that bitmap, not the bitmap itself nor `maxRelayerId`. An off-chain system that tries to mirror exact committee membership purely from the event stream cannot do so; it learns how many relayers are active, not which ones, and must fall back to the update calldata or the active-set view functions. The full data is recoverable on-chain, so on-chain committee selection is unaffected.

Files:

- `ActiveSetLogic::updateActiveSet` - `src/logic/ActiveSetLogic.sol:246-247`

Impact: Off-chain consumers cannot reconstruct exact active-set membership from the event stream alone and must read calldata or call view functions instead. This is an observability gap rather than a consensus break; no funds are at risk and on-chain selection is unaffected.

Recommended Mitigation: Emit the full `activeBitmap` (and `maxRelayerId`) in `ActiveSetUpdated` alongside the existing fields, so an event-only consumer can mirror exact committee membership without reading calldata or calling view functions.

Highway: Acknowledged. Full active-set membership is available through `getCurrentActiveSet` / `getActiveSetByEpoch`, and the relayer already consumes exact membership from those view calls. `ActiveSetUpdated` is used only as a change marker. Emitting the full bitmap would add recurring gas cost for data not consumed from the event stream.

Cyfrin: Rationale accepted: exact membership is available from the active-set getters while retained and can be archived from the successful update call's bitmap, so using the event as a compact change marker is a reasonable observability/gas tradeoff and does not affect on-chain committee selection.

7.4.10 Unnamed non-aliased imports

Description: Several in-scope files import entire files with bare `import "..."` rather than named imports (`import {X} from "..."`). Named imports make the imported symbols explicit, avoid pulling unused symbols into scope, and are the convention already used by the rest of the codebase (e.g. `Authorized.sol`, all `logic/` files).

```
Authorization.sol
4:import "@openzeppelin/contracts/access/AccessControl.sol";
5:import "@openzeppelin/contracts/utils/Pausable.sol";

Storage.sol
10:import "@openzeppelin/contracts/utils/Pausable.sol";

Escrow.sol
9:import "@openzeppelin/contracts/utils/Pausable.sol";
10:import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
11:import "@openzeppelin/contracts/interfaces/IERC20.sol";
12:import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

FeeCollector.sol
9:import "@openzeppelin/contracts/utils/Pausable.sol";

MessageStorage.sol
9:import "@openzeppelin/contracts/utils/Pausable.sol";

EntryLogicProxy.sol
4:import "@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";

ExecutorLogicProxy.sol
4:import "@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";

ActiveSetProxy.sol
4:import "@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";

RelayerRegistryProxy.sol
4:import "@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
```

Recommended Mitigation: Convert to named imports, e.g. `import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";`.

Highway: Fixed in [f135cc0](#), [a76409c](#) and [030a43e](#).

Cyfrin: Verified.

7.4.11 Unused deprecated `TokenConfig` struct

Description: `BridgeTypes.TokenConfig` is explicitly marked `DEPRECATED` ("kept for reference only") and is referenced nowhere in the codebase (a search for `TokenConfig` outside the declaration returns only `ChainToken-`

Config / ChainTokenConfigInput, which are distinct types). It is dead code that adds maintenance noise and a misleading 11-field struct alongside the live config types.

```
BridgeTypes.sol
81:    /// @dev DEPRECATED: Old protocol token configuration - kept for reference only
82:    /// Use GlobalTokenInfo + ChainTokenConfig instead
83:    struct TokenConfig {
...
97:    }
```

Recommended Mitigation: Remove the TokenConfig struct.

Highway: Fixed in [878a34d](#).

Cyfrin: Verified.

7.4.12 aggregatePubkeys returns its single input unvalidated when $n == 1$, skipping the on-curve check it applies for $n \geq 2$

Description: [BLS12381.aggregatePubkeys](#) seeds the accumulator with the first key and then folds in the rest with the G1ADD precompile:

```
aggregated = new bytes(G1_POINT_SIZE);
// copy pubkeys[0] into `aggregated` verbatim (no precompile)
assembly { let src := add(mload(add(pubkeys, 32)), 32) ... mcopy(dst, src, 128) }

for (uint256 i = 1; i < n;) {
    bytes memory pk = pubkeys[i];
    if (pk.length != G1_POINT_SIZE) revert InvalidG1PointLength();
    assembly { ... success := staticcall(gas(), 0x0b, inputPtr, 256, add(aggregated, 32), 128) } //
    ↪ G1ADD
    if (!success) revert G1AddFailed();
    ...
}
```

Per EIP-2537, G1ADD (0x0b) **on-curve-checks both of its inputs** and reverts on a malformed/off-curve point. So for $n \geq 2$, pubkeys[0] is validated transitively, it is an operand of the first G1ADD(pubkeys[0], pubkeys[1]). But for $n == 1$, the loop body never executes: pubkeys[0] is mcopy'd into aggregated and **returned verbatim, with no G1ADD call and therefore no on-curve / subgroup / infinity validation whatsoever**.

The function's validation guarantee is thus **inconsistent with itself**: it validates the lone key for $n \geq 2$ but not for $n == 1$. As a reusable BLS primitive, aggregatePubkeys([P]) should return a validated aggregate; instead it returns P unchecked.

Impact: Informational. Not reachable in Highway's current committee path, but a real correctness gap of the library function:

- **Not exploitable in the shipped flow.** The only caller is ExecutorLogic.verifyInboundProof (step 13), which requires signerCount $\geq$ COMMITTEE_THRESHOLD (87) and blsPubkeys.length == signerCount, so $n \geq 87$ always. aggregatePubkeys is internal, so there is no external $n == 1$ entry point today. And even if reached, the result feeds the PAIRING precompile (which subgroup-checks it) and the keys are registry-bound + PoP-verified.
- **But the primitive is unsound for $n == 1$.** Any future caller, a smaller-quorum configuration, a different aggregation use, a refactor that aggregates a single key, or anyone treating BLS12381 as a general library, that trusts the returned aggregate *without* an independent pairing/subgroup check would accept an off-curve, non-subgroup, or infinity point. The fact that the $n \geq 2$ path *does* validate makes this an easy correctness assumption to make and get wrong.

This is the same class as the general "validation delegated to the pairing" pattern but sharper: here even the on-curve check that G1ADD provides for $n \geq 2$ is silently skipped for $n == 1$.

Proof of Concept: Runnable in [test/boundary/Bls12381Vectors.t.sol](#) (the aggregatePubkeys group), against the **live EIP-2537 precompiles** (default osaka evm). The contrast pair below feeds the *same* off-curve point $(x, y) = (1, 1)$, a valid 128-byte encoding (so it clears the length check) that fails $y^2 = x^3 + 4$, and shows it is **accepted at $n = 1$ but reverts at $n = 2$:**

```

/// Off-curve G1 point: (x, y) = (1, 1), which fails y^2 = x^3 + 4. Valid 128-byte
/// EIP-2537 encoding, so it passes the length check but NOT the on-curve check.
function _offCurve() internal pure returns (bytes memory p) {
    p = new bytes(128);
    p[63] = 0x01; // x = 1
    p[127] = 0x01; // y = 1
}

/// External wrapper so `vm.expectRevert` can catch reverts from the internal lib fn.
function aggExternal(bytes[] memory pks) external view returns (bytes memory) {
    return BLS12381.aggregatePubkeys(pks);
}

/// L-08 PoC: n == 1 returns the lone key VERBATIM with NO precompile validation -
/// an off-curve point is accepted (no `G1ADD`, no on-curve check, no revert).
function test_L08_aggregate_n1_returnsOffCurveKeyUnvalidated() public view {
    bytes[] memory pks = new bytes[](1);
    pks[0] = _offCurve();
    assertEq(BLS12381.aggregatePubkeys(pks), _offCurve(), "n==1 returns input unvalidated (L-08)");
}

/// L-08 contrast: the SAME off-curve key REVERTS for n == 2 - the first `G1ADD`
/// on-curve-checks it. Identical input, validated at n=2, accepted at n=1.
function test_L08_aggregate_n2_offCurveKeyReverts() public {
    bytes[] memory pks = new bytes[](2);
    pks[0] = _offCurve();
    pks[1] = _pk0(); // any valid key
    vm.expectRevert(BLS12381.G1AddFailed.selector);
    this.aggExternal(pks);
}

```

The contrast **is** the finding: identical bad input, validated at $n = 2$, accepted at $n = 1$. The same group also covers the legitimate cases ($n == 1$ valid key returns itself, order-independence, EmptyPubkeyArray / InvalidG1PointLength reverts).

```

$ forge test --match-test test_L08_aggregate
[PASS] test_L08_aggregate_n1_returnsOffCurveKeyUnvalidated()
[PASS] test_L08_aggregate_n2_offCurveKeyReverts()

```

Recommended Mitigation: aggregatePubkeys does not validate its input when $n == 1$: it returns the lone key verbatim, without the on-curve check the $n \geq 2$ path applies. Document this, and do not call aggregatePubkeys with a single key. A caller that might pass one key should guard against $n == 1$ or validate the key independently (on-curve / subgroup / non-infinity) before trusting the returned aggregate. The shipped committee path always has $n \geq 87$, so this is a usage note for the library function, not a change to the current flow.

Highway: Fixed in [1cf29d0](#).

Cyfrin: Verified.

7.5 Gas Optimization

7.5.1 Redundant `hasTokenChainConfig` external call before `getTokenChainConfig`

Description: `EntryLogic::_processFunds` issues two external calls to `Storage` - `hasTokenChainConfig(tokenId, targetChain)` then `getTokenChainConfig(tokenId, targetChain)` - but `getTokenChainConfig` already reverts with `TokenNotConfiguredForChain` when the config does not exist (`Storage.sol:493-495`). The pre-check call is pure overhead (~2,600 gas for the extra `CALL` plus the duplicate `tokenChainConfigs[...].exists` SLOAD) on the user-facing outbound bridge hot path. The revert path is identical (`TokenNotConfiguredForChain`).

```
EntryLogic.sol
611:         if (!_storage().hasTokenChainConfig(tokenId, targetChain)) {
612:             revert BridgeErrors.TokenNotConfiguredForChain(tokenId, targetChain);
613:         }
614:         BridgeTypes.ChainTokenConfig memory chainConfig = _storage().getTokenChainConfig(tokenId,
↳ targetChain);
```

Recommended Mitigation: Drop the `hasTokenChainConfig` pre-check and call `getTokenChainConfig` directly - it already reverts `TokenNotConfiguredForChain` for the missing-config case:

```
BridgeTypes.ChainTokenConfig memory chainConfig = _storage().getTokenChainConfig(tokenId, targetChain);
```

Highway: Fixed in [7fda4e5](#).

Cyfrin: Verified.

7.5.2 Cache the `EntryLogic::_storage` slot accessor instead of re-reading proxy slot on every call

Description: Each call to the `_storage` accessor executes `sload(STORAGE_SLOT)` to fetch the `IStorage` address from the proxy's storage slot 1. The entry functions call it 3-4 times per invocation (once per external call). Because these are loads of a constant slot punctuated by external `CALLs` to other contracts (the optimizer cannot prove an opaque external call left slot 1 untouched, and `via_ir` does not hoist `sload` across call boundaries), each one re-issues a warm SLOAD (~100 gas each). Caching the address into one local on each user-facing path removes 2-3 SLOADs per bridge call. The same applies to `_feeCollector` (read at `EntryLogic.sol:191` and again at `EntryLogic.sol:282`).

```
EntryLogic.sol
385:         BridgeTypes.ChainInfo memory chainInfo = _storage().getChainInfo(targetChain);
395:         uint128 nonce = _storage().getAndUpdateNonce(targetChain);
403:         tokenInfo = _storage().getToken(tokenId);
464:         BridgeTypes.ChainInfo memory chainInfo = _storage().getChainInfo(targetChain);
474:         uint128 nonce = _storage().getAndUpdateNonce(targetChain);
533:         BridgeTypes.ChainInfo memory chainInfo = _storage().getChainInfo(targetChain);
539:         uint128 nonce = _storage().getAndUpdateNonce(targetChain);
551:         BridgeTypes.GlobalTokenInfo memory tokenInfo = _storage().getToken(tokenId);
611:         if (!_storage().hasTokenChainConfig(tokenId, targetChain)) {
614:         BridgeTypes.ChainTokenConfig memory chainConfig = _storage().getTokenChainConfig(tokenId,
↳ targetChain);
```

Recommended Mitigation: Read the accessor once at the top of each entry function and pass the local through (including into `_processFunds`, so its internal `_storage` reads are eliminated):

```
IStorage stor = _storage();
BridgeTypes.ChainInfo memory chainInfo = stor.getChainInfo(targetChain);
// ...
uint128 nonce = stor.getAndUpdateNonce(targetChain);
```

Highway: Fixed in [eddbc78](#), [5a6746b](#) and [33e1cf3](#).

Cyfrin: Verified.

7.5.3 Storage::getAndUpdateNonce increments storage then re-reads the same slot

Description: Storage::getAndUpdateNonce writes nonces[chainId]++ and then returns nonces[chainId], issuing a second SLOAD of a slot it just wrote. The optimizer cannot forward the just-stored value through the SSTORE, so the return re-reads it (~100 warm-SLOAD gas). This runs once per outbound bridge call (hot path).

```
Storage.sol
423:     function getAndUpdateNonce(uint32 chainId) external onlyRole(NONCE_MANAGER_ROLE) returns
    ↪ (uint128 nonce) {
424:         nonces[chainId]++;
425:         return nonces[chainId];
426:     }
```

Recommended Mitigation: Compute the new value once into a local, store it, and return the local:

```
function getAndUpdateNonce(uint32 chainId) external onlyRole(NONCE_MANAGER_ROLE) returns (uint128
    ↪ nonce) {
    nonce = nonces[chainId] + 1;
    nonces[chainId] = nonce;
}
```

Highway: Fixed in [cdc44db](#) and [75427ef](#).

Cyfrin: Verified.

7.5.4 Suboptimal storage packing on mapping-resident structs

Description: The compiler lays struct fields out in declaration order and never reorders them. Two storage-resident structs place a 1-byte field after a full-slot type, forcing it into its own slot when it could share a partially-filled slot. Neither struct is abi.encode-passed by value nor pinned by a *_TYPEHASH string, so reordering is safe.

1. BridgeTypes.Relayer - stored in mapping(uint32 => Relayer) _relayers (RelayerRegistryStorage.sol:24), one instance per relayer (cap MAX_RELAYER_ID = 6000). Current layout is 5 slots: id(4)+manager(20)=slot0; beneficiary(20)=slot1; blsPublicKey(bytes)=slot2; blsKeyHash(bytes32)=slot3; exists(bool)=slot4. Moving exists next to beneficiary packs it into slot1 (20+1 bytes), saving 1 slot per relayer.

```
BridgeTypes.sol
118:     struct Relayer {
119:         uint32 id;
120:         address manager;
121:         address beneficiary;
122:         bytes blsPublicKey;
123:         bytes32 blsKeyHash;
124:         bool exists;
125:     }
```

2. BridgeTypes.ActiveRelayerSet - stored in the fixed buffer ActiveRelayerSet[10] _activeRelayerSets (ActiveSetStorage.sol:32), 10 instances. Current layout is 5 slots: epoch(4)=slot0; startBlock(uint256)=slot1; activeBitmap(bytes)=slot2; maxRelayerId(4)=slot3; epochRandomness(bytes32)=slot4. Moving maxRelayerId adjacent to epoch packs both uint32s into slot0 (8 bytes), saving 1 slot per entry (10 slots across the buffer).

```
BridgeTypes.sol
128:     struct ActiveRelayerSet {
129:         uint32 epoch;
130:         uint256 startBlock;
131:         bytes activeBitmap;
132:         uint32 maxRelayerId;
133:         bytes32 epochRandomness;
134:     }
```

Recommended Mitigation: Reorder so the sub-32-byte field clusters with another sub-slot field:

```

struct Relayer {
    uint32 id;           // slot 0
    address manager;     // slot 0 (24 bytes)
    address beneficiary; // slot 1
    bool exists;         // slot 1 (21 bytes) - packed
    bytes blsPublicKey;  // slot 2
    bytes32 blsKeyHash;  // slot 3
}

struct ActiveRelayerSet {
    uint32 epoch;        // slot 0
    uint32 maxRelayerId; // slot 0 (8 bytes) - packed
    uint256 startBlock;  // slot 1
    bytes activeBitmap;  // slot 2
    bytes32 epochRandomness; // slot 3
}

```

Note: ActiveSetStorage and RelayerRegistryStorage are deployed once and treated as permanent addresses (not behind a proxy and not upgraded), so the reorder is safe at deployment; do not apply it as a layout change to already-populated live storage.

Highway: Fixed in [9f318ec](#).

Cyfrin: Verified.